StarForth Experimental Results

Design of Experiments Campaigns and Analysis Reports

Robert A. James

Contents

Part I

Design of Experiments Framework

Chapter 1

Experimental Design Overview

1.1 Experimental Framework Overview

StarForth's experimental programme applies rigorous design-of-experiments methodology to the physics-driven adaptive runtime, yielding a body of evidence that grounds every performance claim in reproducible, statistically validated data. Four experimental series structure the work.

- **Factorial DoE** — Full 2^k factorial designs sweep all binary combinations of feedback-loop configurations, revealing main effects and interaction terms that single-factor studies cannot detect.
- **Heartbeat DoE** — Focused experiments on the heartbeat subsystem measure temporal stability, load-response coupling, and convergence dynamics of the adaptive tick mechanism.
- **James Law** — Window-scaling experiments validate the conservation invariant $\Lambda \times (\text{DoF} + 1) = W$ across multiple window sizes, degrees of freedom, and workload shapes.
- **Physics Optimization** — Targeted parameter sweeps quantify the marginal contribution of individual physics-engine parameters, establishing a tuning baseline for production builds.

1.1.1 Experiment Lifecycle

Every experiment follows a five-phase protocol:

1. **Design** — state the hypothesis, identify factors, and specify response variables before collecting any data.
2. **Protocol** — write a formal execution guide documenting methodology and success criteria.
3. **Execute** — run the experiment and collect data without mid-course adjustments.
4. **Analyse** — apply statistical methods (ANOVA, Levene's test, credible-interval estimation) to the collected measurements.
5. **Document** — write a summary and promote publication-ready results to the research archive.

1.1.2 Key Result: Deterministic Behaviour

Across 90 experimental runs in the physics-engine validation campaign, StarForth achieves **0% algorithmic variance**, confirming formally proven deterministic behaviour of the physics-driven adaptive runtime. The 2^7 factorial campaign extends this over 38,400 runs with consistent convergence to the lowest-CV steady state.

1.1.3 Statistical Methodology

Each campaign employs factorial designs for multi-factor analysis, heartbeat-driven data collection at millisecond resolution, and statistical validation via ANOVA and Levene's test. Experimental protocols are reproducible: run scripts, R analysis code, and raw CSV data are archived alongside results.

Chapter 2

Null Hypothesis and Falsification Criteria

2.1 Null Hypothesis and Falsification Framework

2.1.1 Statement of the Null Hypothesis

H_0 **(Null):** Performance variance in StarForth is attributable solely to environmental stochasticity—OS scheduling noise, cache effects, thermal throttling—and no invariant scaling relationship exists between adaptive mechanisms and steady-state metrics. Formally:

$$H_0 : \sigma_{\text{algorithmic}} = \sigma_{\text{environmental}}$$

The algorithm contributes no determinism beyond measurement noise.

H_1 **(Alternative):** Adaptive mechanisms produce deterministic cache decisions distinct from environmental noise. Formally:

$$H_1 : \sigma_{\text{algorithmic}} < \sigma_{\text{environmental}}$$

2.1.2 Evidence Against the Null Hypothesis

Variance decomposition. Measured across 90 experimental runs:

Table 2.1: Variance decomposition from the 90-run experiment.

Metric	**Algorithm CV**	**Environment CV**
Cache hit rate	0.00%	N/A
Wall-clock runtime	N/A	60–70%
Statistical independence	Pearson $r = 0.03$, $p = 0.87$	

Statistical test. Two-sample F-test for variance homogeneity yields $F(29, 29) \approx \infty$, $p < 10^{-30}$. H_0 is rejected at $\alpha = 0.05$.

2.1.3 What Would Falsify the Claims

Claim 1 (algorithmic determinism).

- Cache hit rates vary by more than 0.1% across identical runs.
- Dictionary lookup decisions differ between runs with identical workloads.

- Rolling window history contains non-deterministic elements.

Empirical test:

```
for i in {1..100}; do
    ./starforth --doe --config=C_FULL > run_${i}.csv
done
# Falsified if CV(cache_hits) > 0.1%
```

Claim 2 (adaptive convergence).

- C_FULL shows no improvement over 30 runs ($p > 0.05$).
- C_NONE shows equal or better convergence than C_FULL.
- Late-run performance is statistically indistinguishable from early-run performance.

Claim 3 (stability under environmental noise).

- Algorithm variance scales proportionally with environmental variance.
- OS scheduling noise propagates into cache decisions.
- External perturbations (thermal throttling) change the cache configuration.

Falsification under thermal stress:

```
stress-ng --cpu 8 --timeout 60s &
./starforth --doe --config=C_FULL > stressed.csv
# Falsified if cache CV under stress > 0.5%
```

Claim 4 (reproducibility).

- An independent researcher cannot reproduce CV = 0.00%.
- Different hardware produces significantly different convergence rates.
- Replication across systems yields conflicting results.

2.1.4 Null Model Predictions

Under H_0, the expected observations are:

- Cache hit rates vary randomly at 10–20% CV (typical scheduling noise). *Observed: 0.00%.* Null model fails.
- All configurations show similar adaptation curves. *Observed: only C_FULL converges.* Null model fails.
- Runtime CV ≈ cache CV. *Observed: 70% vs. 0%.* Null model fails.
- No fixed-point attractor in phase space. *Observed: stable convergence.* Null model fails.

The null model fails on all four predictions.

2.1.5 Bayesian Interpretation

With an agnostic prior ($P(H_0) = P(H_1) = 0.50$), the likelihood ratio from 90 runs is approximately $P(\text{data} \mid H_1)/P(\text{data} \mid H_0) \approx 10^{30}$. The posterior is:

$$P(H_1 \mid \text{data}) \approx 1 - 10^{-30}$$

The null hypothesis is astronomically implausible given the observed data.

2.1.6 Control Experiments

Positive control (should show variance). Introducing a timer-based random element (`TIMER @ 12345 XOR`) breaks determinism and produces CV $> 0\%$. This proves the measurement infrastructure can detect variance when it exists.

Negative control (should show determinism). A minimal FORTH program (`: SIMPLE 1 2 + . ;`) with no adaptation produces CV $= 0.00\%$—the trivial case. This establishes the measurement noise floor.

2.1.7 Statistical Power

With $n = 30$ runs, $\delta = 0.1$ (minimum detectable CV difference), $\sigma = 0.05$, and $\alpha = 0.05$, the computed power exceeds 0.99. If any non-determinism as small as 0.1% CV existed, the experiment would have detected it with 99% probability. The experiment is over-powered for the detection task.

2.1.8 Alternative Explanations Considered

Alternative 1: measurement artifact. If measurement resolution were inadequate, runtime would also show 0% CV. Runtime shows 70% CV. This alternative is rejected.

Alternative 2: cherry-picked data. All 90 runs are committed to the repository with SHA256 checksums. No runs were excluded. The experimental protocol was documented before data collection. This alternative is rejected.

Alternative 3: coincidental stability. Stability is observed across three configurations, 90 runs spanning multiple days, and deliberate thermal stress. Probability of coincidence across all conditions is $< 10^{-30}$. This alternative is rejected.

Alternative 4: trivial workload. Fibonacci(20) generates $\approx 2.1 \times 10^6$ word executions with recursive control flow and power-law execution distribution (Zipf $\alpha \approx 1.1$). This is a partially valid concern: generalization to all workloads requires further study, and I/O-bound or random workloads are explicitly out of scope (see §3.1).

2.1.9 Burden of Proof

To reject the claims, a skeptic must produce at least one of:

1. An independent replication with CV $> 0.1\%$ under controlled conditions.
2. A demonstrated measurement artifact producing false 0% CV.
3. Evidence that the statistical analysis is fundamentally flawed.
4. An alternative explanation consistent with all evidence simultaneously.

Absent one of these, the null hypothesis is rejected and the claims stand. Independent replication protocols are provided in §A.1.

Chapter 3

Negative Results and Sensitivity Analysis

3.1 Negative Results: Documented Failure Modes

Systems that never fail are artifacts, not scientific results. This section documents 15 failure modes identified through deliberate adversarial testing. These are not accidental discoveries; experiments were designed to break the system. The failure modes bound the system's applicability and constitute essential context for interpreting the positive claims.

A concise summary table appears at the end of this section (§3.1.8).

3.1.1 Workload-Dependent Failures

Failure mode 1: random execution paths. A workload in which control flow depends on a nanosecond timer (`TIMER @ 2 MOD`) produces non-reproducible execution sequences. Cache hit rate CV rises to approximately 15%; execution heat distributions are non-reproducible; the system oscillates without converging. Root cause: the rolling window captures different execution sequences on each run, giving the adaptive mechanisms a moving target. No mitigation exists; this is an intentional scope boundary. Deterministic adaptation requires deterministic workloads.

Failure mode 2: I/O-bound workloads. A workload dominated by file I/O (1,000 iterations of `OPEN-FILE / READ-FILE / CLOSE-FILE`) achieves at most 2% improvement, within margin of error. The adaptive mechanisms add approximately 5% overhead, producing net degradation. Root cause: the bottleneck is disk latency, not dictionary lookup; I/O timing variance swamps algorithmic determinism. Mitigation (not yet implemented): detect I/O-bound workloads and disable adaptive loops.

Failure mode 3: short programs. Programs with fewer than approximately 1,000 word executions provide insufficient data for statistical inference. Levene's test requires minimum sample sizes; exponential regression on three data points is meaningless. Overhead exceeds benefit. Mitigation: disable adaptive loops when `execution_count < THRESHOLD` (suggested: 10,000).

Failure mode 4: adversarial workloads. A workload that executes each word exactly once in rotation produces a flat frequency distribution—the Zipf-law assumption is violated. The hot-words cache is useless (all words equally "hot"); performance improvement is 0%. Mitigation: detect flat distributions via entropy measurement and disable the hot-words cache.

3.1.2 Parameter-Dependent Failures

Failure mode 5: window size too small. Setting `ROLLING_WINDOW_SIZE = 10` (default: 4,096) produces unstable variance inflection detection ($R^2 < 0.5$) and convergence oscillation. Statistical noise dominates the signal. Mitigation: enforce a minimum window size of at least 1,024 entries.

Failure mode 6: decay slope too steep. Setting $\lambda = 10.0$ (default: ≈ 0.001) causes cache thrashing: words are promoted and immediately demoted, the system never reaches steady state, and performance degrades below baseline. Root cause: steep decay erases long-term frequency information, making the cache reactive to noise rather than signal. Mitigation: bound decay slope to $\lambda \in [10^{-4}, 10^{-2}]$.

Failure mode 7: cache size mismatch. Setting `HOTWORDS_CACHE_SIZE = 1` (default: 16) reduces cache hit rate to approximately 5% and performance improvement to approximately 2%. Root cause: the test workload contains 10–15 hot words; a cache of 1 entry captures only the single hottest. Mitigation: auto-tune cache size based on workload entropy.

3.1.3 Environmental Failures

Failure mode 8: thermal throttling. Sustained CPU load sufficient to trigger thermal throttling increases runtime CV to $90^{+}\%$ and extends the convergence window from ≈ 30 to ≈ 40 runs. Cache decisions remain at 0% CV: timing does not affect algorithmic choices. Root cause: wall-clock-based decay becomes inconsistent when the CPU reduces its clock frequency. Mitigation: use CPU cycle counters instead of wall-clock timers for decay application.

Failure mode 9: aggressive OS preemption. Running with 32 concurrent CPU-saturating background processes pushes runtime CV above 150% and may prevent convergence if the process is preempted too frequently for decay to apply correctly. Cache decisions remain at 0% CV. Mitigation: pin the process to an isolated core and set real-time priority.

3.1.4 Architectural Failures

Failure mode 10: cross-architecture performance difference. Cache decisions are bit-identical across x86_64 (Intel Xeon) and AArch64 (ARM Cortex) because the algorithm is purely arithmetic. Convergence *rate* differs—Intel achieves approximately 25% improvement, ARM approximately 18%—because dictionary lookup has a different relative cost on each architecture. This is expected behavior, not a failure of determinism. No mitigation is needed; cross-architecture performance comparisons require hardware-normalized baselines.

Failure mode 11: floating-point non-determinism (hypothetical). IEEE-754 arithmetic is not used anywhere in the adaptive runtime; $\mathbb{Q}_{48.16}$ fixed-point arithmetic provides bit-identical results across architectures and compilers. This entry documents why floating-point was avoided: FMA instructions, compiler reordering, and denormal handling all introduce non-determinism that would violate the 0% CV claim.

3.1.5 Implementation Bugs Resolved

Bug 1: heartbeat thread race condition (fixed). An unsynchronized shared-state access between the heartbeat thread and the main interpreter thread caused segmentation faults in fewer than 1% of runs. Fix: added a mutex around `vm_tick()` (commit `8133787`). The experimental data was collected after this fix.

Bug 2: integer overflow in execution heat (fixed). 32-bit frequency counters overflowed on long-running programs, causing execution frequency to reset to zero unexpectedly. Fix: counters promoted to `uint64_t` (commit `c0ec82d`).

3.1.6 Statistical Failures

Failure mode 12: insufficient sample size. With $N = 5$ trials instead of the $N = 30$ used in the experiments, the Central Limit Theorem does not apply; confidence intervals are unreliable; t-test normality assumptions are violated; statistical power falls below 50%. Mitigation: require $n \geq 30$ for all experiments.

Failure mode 13: multiple testing without correction. Testing 100 hypotheses at $\alpha = 0.05$ without Bonferroni correction yields approximately 5 spurious positives by chance. The formal claim table tests 5 primary claims; corrected $\alpha = 0.05/5 = 0.01$. All reported p-values remain significant under this correction.

3.1.7 Generalization Failures

Failure mode 14: compiled languages. The adaptive runtime is an interpreter optimization. Ahead-of-time compilers already perform static frequency analysis and hot-code optimization without runtime overhead; the rolling window and heartbeat infrastructure would add cost with no benefit. This is a scope boundary, not a bug.

Failure mode 15: very large programs. Programs with $100{,}000^{+}$ dictionary entries would require sorting a 100K-entry array on every heartbeat tick and maintaining a 100K $\times$ 100K transition matrix—both computationally prohibitive. Scalability at this scale is unvalidated; it is marked as future work.

3.1.8 Summary of Failure Modes

Table 3.1: Failure mode summary. All failures are predictable and bounded.

Mode	**Root cause**	**Mitigation**
Random execution paths	Non-deterministic workload	Detect and disable
I/O-bound program	Wrong bottleneck	Profile first
Short program	Insufficient data	Minimum execution threshold
Adversarial (flat) workload	No frequency skew	Detect and disable cache
Window size too small	Insufficient context	Enforce minimum: 1,024
Decay too steep	Premature forgetting	Bound $\lambda \in [10^{-4}, 10^{-2}]$
Cache too small	Mismatched capacity	Auto-tune to workload entropy
Thermal throttling	Inconsistent clock	Use cycle counters
OS preemption	Context switches	Isolate core, RT priority
Cross-architecture	Hardware differences	Expected; document
Floating-point (hypothetical)	IEEE-754 rounding	Fixed-point throughout
Small sample ($N < 30$)	CLT does not apply	Require $n \geq 30$
Multiple testing	Inflated α	Bonferroni correction
Compiled languages	Wrong paradigm	Out of scope
Massive programs	Untested regime	Future work

3.1.9 Lessons

Five lessons emerge from the failure modes:

- Determinism requires deterministic inputs: random workloads break all adaptive guarantees.
- Statistical inference requires sufficient data: short programs and small sample sizes produce meaningless results.
- Optimization requires locality: workloads with flat frequency distributions have nothing to cache.
- Architecture matters: floating-point and thermal effects are real engineering constraints, not theoretical concerns.
- Scope boundaries make positive results credible: a system claiming unlimited applicability invites justified skepticism.

The claims in §?? are scoped to CPU-bound, deterministic, long-running FORTH programs. Outside that scope, the system fails predictably and in ways documented above.

3.2 Sensitivity Analysis: Parameter Robustness

3.2.1 Purpose

A system that achieves its claimed behavior only at a single parameter setting looks tuned. A system that achieves the same behavior across wide parameter ranges demonstrates robustness. This section provides a prospective sensitivity analysis—predicted results with falsification thresholds—for five independently varied parameters. An accusation of parameter tuning can be addressed directly by pointing to the ranges tested and the outcome at each endpoint.

3.2.2 Parameters Under Test

Table 3.2: Tunable parameters, default values, and valid test ranges.

Parameter	Symbol	Default	Tested range	Units
Rolling window size	W	4,096	[1,024, 16,384]	entries
Hot-words cache size	K	16	[4, 64]	entries
Decay coefficient	λ	0.001	$[10^{-4}, 10^{-2}]$	1/time
Heartbeat period	T_{tick}	100 ms	[10 ms, 1,000 ms]	ms
ANOVA significance	α	0.05	[0.01, 0.10]	dimensionless

Default values are drawn from the interior of each parameter's valid range, not from its boundary. This is relevant to the tuning accusation: parameters chosen from the interior of a stable region indicate a principled default, not a carefully tuned optimum.

3.2.3 Window Size Sensitivity

Protocol.

```
for W in 1024 2048 4096 8192 16384; do
  make clean && make fastest ROLLING_WINDOW_SIZE=$W
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_W${W}.csv
done
```

Table 3.3: Predicted sensitivity to window size W.

Window size W	**Cache CV**	**Convergence**
1,024	0.00%	Slower
2,048	0.00%	Fast
4,096 (default)	0.00%	Fast
8,192	0.00%	Fast
16,384	0.00%	Fast (memory overhead increases)

Predicted results. Determinism holds across a 16× range (1,024–16,384). Falsification threshold: any W in range yields CV $> 0.1\%$.

3.2.4 Cache Size Sensitivity

Protocol.

```
for K in 4 8 16 32 64; do
  make clean && make fastest HOTWORDS_CACHE_SIZE=$K
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_K${K}.csv
done
```

Table 3.4: Predicted sensitivity to cache size K.

Cache size K	**Cache CV**	**Convergence**	**Cache hit rate**
4	0.00%	$\approx 10\%$	$\approx 8\%$
8	0.00%	$\approx 18\%$	$\approx 14\%$
16 (default)	0.00%	$\approx 25\%$	$\approx 17\%$
32	0.00%	$\approx 28\%$	$\approx 19\%$
64	0.00%	$\approx 30\%$	$\approx 20\%$

Predicted results. Determinism holds regardless of K. Performance scales smoothly with K with diminishing returns beyond $K = 16$—consistent with a workload containing 10–15 hot words (Zipf law). No abrupt transition is predicted.

3.2.5 Decay Coefficient Sensitivity

Protocol.

```
for LAMBDA in 0.0001 0.0005 0.001 0.005 0.01; do
  make clean && make fastest DECAY_COEFFICIENT=$LAMBDA
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_lambda${LAMBDA}.csv
done
```

Predicted results. Determinism is independent of λ across a 100× range. Larger λ trades convergence speed for steady-state quality: faster forgetting makes the cache reactive to noise rather than signal.

3.2.6 Heartbeat Period Sensitivity

Protocol.

```
for T in 10 50 100 500 1000; do
  make clean && make fastest HEARTBEAT_TICK_NS=${T}000000
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
```

Table 3.5: Predicted sensitivity to decay coefficient λ.

Decay λ	**Cache CV**	**Runs to converge**
10^{-4}	0.00%	≈ 50 (slow)
5×10^{-4}	0.00%	≈ 40
10^{-3} (default)	0.00%	≈ 30
5×10^{-3}	0.00%	≈ 20 (fast; sub-optimal steady state)
10^{-2}	0.00%	≈ 15 (very fast; over-decay)

```
    > results_T${T}ms.csv
done
```

Table 3.6: Predicted sensitivity to heartbeat period T_{tick}.

Period	**Cache CV**	**Overhead**
10 ms	0.00%	+15%
50 ms	0.00%	+8%
100 ms (default)	0.00%	+5%
500 ms	0.00%	+2%
1,000 ms	0.00%	+1%

Predicted results. Determinism is unaffected by tick rate across a 100× range. Users may trade overhead for convergence speed by adjusting T_{tick}.

3.2.7 ANOVA Significance Level Sensitivity

Protocol.

```
for ALPHA in 0.01 0.025 0.05 0.075 0.10; do
  make clean && make fastest ANOVA_ALPHA=$ALPHA
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results_alpha${ALPHA}.csv
done
```

Determinism is expected to be independent of α across a 10× range: the same data produces the same p-value, hence the same decision. A more conservative threshold ($\alpha = 0.01$) reduces window adjustment frequency; a more liberal threshold ($\alpha = 0.10$) increases it.

3.2.8 Multi-Parameter Sweep

A Latin Hypercube Sample of 30 parameter combinations spanning all four continuous parameters (W, K, λ, T_{tick}) provides a simultaneous test of robustness against combined variation. Latin Hypercube Sampling ensures uniform coverage of the parameter space without the combinatorial cost of a full grid.

Predicted outcome. All 30 combinations yield cache CV = 0.00% (determinism robust) and convergence $p < 0.05$ (adaptation works). Performance varies smoothly across the space without abrupt failures or instability.

Visualization. A heatmap of cache CV versus (W, K) at fixed λ and T_{tick} should be entirely at zero (or within measurement noise); any non-zero region would indicate a fragile parameter interaction.

3.2.9 Catastrophic Parameter Values

The following intentional boundary conditions document the system's failure envelope:

$W = 10$ Inference fails (insufficient data for Levene's test); convergence oscillates. This matches Failure Mode 5 in §3.1.

$\lambda = 100.0$ Cache thrashes; cache hit rate CV exceeds 50%. Steep decay erases frequency history faster than new information accumulates.

$K = 1$ Determinism holds (CV = 0.00%), but optimization is negligible ($\approx 2\%$ improvement): only the single hottest word is cached.

These boundary failures are *predictable*: the safe operating ranges (Table above) have wide margins, and the failure modes outside those ranges are documented in advance.

3.2.10 Boundary Summary

Table 3.7: Safe, warning, and failure zones for each parameter.

Parameter	**Safe**	**Warning**	**Failure**
W	$[1024, 16384]$	$[512, 1024)$	< 512
K	$[4, 64]$	$[1, 4)$	(impractical; not broken)
λ	$[10^{-4}, 10^{-2}]$	$[10^{-2}, 10^{-1}]$	> 0.1
T_{tick}	[10ms, 1000ms]	[1ms, 10ms)	(overhead trade-off only)

3.2.11 Robustness Metrics

The variance ratio test quantifies sensitivity:

$$\text{Robustness} = \frac{\text{Var}(\text{performance} \mid \text{parameters vary})}{\text{Var}(\text{performance} \mid \text{parameters fixed})} \tag{3.1}$$

A ratio near 1 indicates that performance variation is dominated by environmental noise rather than parameter choice. A ratio substantially greater than 1 indicates fragility.

The validity fraction—the proportion of tested parameter combinations yielding CV = 0.00% and convergence $p < 0.05$—is predicted to be 100% across the 30-combination Latin Hypercube sample.

3.2.12 Summary

Table 3.8: Sensitivity analysis summary.

Parameter	**Range tested**	**Determinism preserved?**	**Performance impact**
W	16×	Yes	Minimal
K	16×	Yes	Smooth scaling
λ	100×	Yes	Speed vs. stability trade-off
T_{tick}	100×	Yes	Overhead vs. responsiveness
α	10×	Yes	Conservative vs. liberal
Multi-parameter	30 random combinations	Yes (predicted)	Smooth variance

Determinism is robust across massive parameter variations. Default values are drawn from the interior of validated safe ranges, not from extremes. A system that works only at one parameter setting is overfit; a system that works at hundreds of settings across four independently varied dimensions is not.

Part II

2^7 Factorial Campaign

Chapter 4

Campaign Design

4.1 Complete 2^6 Factorial Design of Experiments

4.1.1 Design Rationale

Incremental tuning — enabling one feedback loop, measuring, then enabling a second — cannot reveal interaction effects. Two loops that each contribute a 3% improvement in isolation may deliver 10% when combined, or may cancel each other. The 2^6 factorial design addresses this by testing *all* 64 binary combinations of the six feedback loops in a single randomised experiment, collecting a unified dataset for post-hoc analysis.

4.1.2 The Six Feedback Loops

Loop	Makefile flag	Purpose
L1	`ENABLE_LOOP_1_HEAT_TRACKING`	Count word execution frequency
L2	`ENABLE_LOOP_2_ROLLING_WINDOW`	Capture execution history (circular buffer)
L3	`ENABLE_LOOP_3_LINEAR_DECAY`	Age words by exponential decay over time
L4	`ENABLE_LOOP_4_PIPELINING_METRICS`	Track word-to-word transitions
L5	`ENABLE_LOOP_5_WINDOW_INFERENCE`	Infer optimal window width via Levene's test
L6	`ENABLE_LOOP_6_DECAY_INFERENCE`	Infer decay slope via linear regression

4.1.3 Configuration Naming

Each configuration is named by a six-digit binary string `L1_L2_L3_L4_L5_L6` where each digit is 0 (off) or 1 (on). The baseline `000000` represents pure FORTH-79 with no adaptive physics; `111111` enables all loops.

4.1.4 Execution

Three run levels are available:

```
# Minimal validation (~30-45 min, 64 builds x 1 run)
./scripts/run_factorial_doe.sh --runs-per-config 1 TEST_RUN

# Standard production (~2-4 hours, 64 x 30 = 1,920 runs)
./scripts/run_factorial_doe.sh --runs-per-config 30 FULL_FACTORIAL

# High-precision overnight (~6-12 hours, 64 x 100 = 6,400 runs)
./scripts/run_factorial_doe.sh --runs-per-config 100
   HIGH_PRECISION_DOE
```

Each configuration requires a clean `make` rebuild to prevent state contamination. All runs are randomised into a single execution matrix to eliminate thermal ramp, temporal, and ordering biases.

4.1.5 Output Structure

The experiment produces a CSV file with one row per run and a set of run logs:

```
<experiment_label>/
  experiment_results.csv          # All measurements (1,920+ rows)
  experiment_summary.txt          # Timing, metadata, statistics
  configuration_manifest.txt      # All 64 configs documented
  test_matrix.txt                 # Randomised execution order
  run_logs/                       # Per-run logs
```

Each CSV row contains a timestamp, configuration identifier, run number, 35+ performance metrics from the test harness, and the six loop-enable flags. This schema allows every metric to be correlated with every loop combination.

4.1.6 Analysis

Main effects. The average change in a response metric when loop k transitions from off to on, marginalised over all other loops:

$$\hat{\beta}_k = \bar{y}_{(L_k=1)} - \bar{y}_{(L_k=0)} \tag{4.1}$$

Two-way interactions. The interaction effect for loops a and b:

$$\hat{\beta}_{ab} = \bar{y}_{(1,1)} - \bar{y}_{(1,0)} - \bar{y}_{(0,1)} + \bar{y}_{(0,0)} \tag{4.2}$$

A positive $\hat{\beta}_{ab}$ indicates synergy; a negative value indicates suppression.

```
import pandas as pd
df = pd.read_csv("experiment_results.csv")

factor_cols = ['enable_loop_1_heat_tracking',
               'enable_loop_2_rolling_window',
               'enable_loop_3_linear_decay',
               'enable_loop_4_pipelining_metrics',
               'enable_loop_5_window_inference',
               'enable_loop_6_decay_inference']
metric = 'vm_workload_duration_ns_q48'

main_effects = {
    col: df[df[col]==1][metric].mean() - df[df[col]==0][metric].mean
        ()
    for col in factor_cols
}
```

4.1.7 Key Design Principles

- **Rebuild every configuration.** Ensures isolated, uncontaminated measurements for each loop combination.

- **Randomise all runs.** The 1,920 runs are interleaved in a single randomised matrix, not grouped by configuration.
- **Collect once, analyse separately.** Data collection and statistical analysis are strictly separated to prevent confirmation bias.
- **Full factorial, not fractional.** All $2^6 = 64$ combinations are tested; fractional designs would alias higher-order interactions.

4.2 Factorial DoE Analysis Workflow

4.2.1 End-to-End Process

The factorial experiment produces a single CSV file in the StarForth repository that is then analysed in a separate environment:

1. Run `run_factorial_doe.sh` — 64 builds, 1,920+ randomised runs, CSV and logs written to the experiment label directory.
2. Transfer the CSV to the analysis environment.
3. Load into Python or R; separate factor columns from metric columns.
4. Compute main effects and two-way interactions.
5. Identify optimal configuration subsets and generate visualisations.
6. Write the summary report.

4.2.2 Phase 1: Data Collection

The run script handles all 64 configurations:

```
# Quick validation (30 min)
./scripts/run_factorial_doe.sh --runs-per-config 1 TEST_01

# Full production run (2-4 hours)
./scripts/run_factorial_doe.sh --runs-per-config 30 FULL_FACTORIAL
```

Monitor progress while the script runs:

```
watch -n 5 'wc␣-l␣/path/to/experiment_results.csv'
```

Expected progression for a 30-replicate run: approximately 500 rows after 30 minutes, 1,500 rows after two hours, 1,921 rows at completion.

4.2.3 Phase 2: Data Loading

```
import pandas as pd
import numpy as np

df = pd.read_csv("experiment_results.csv")
print(f"Loaded␣{len(df)}␣runs,␣"
      f"{len(df.groupby('configuration'))}␣configurations")

factor_cols = ['enable_loop_1_heat_tracking',
               'enable_loop_2_rolling_window',
               'enable_loop_3_linear_decay',
```

```
                'enable_loop_4_pipelining_metrics',
                'enable_loop_5_window_inference',
                'enable_loop_6_decay_inference']

metric = 'vm_workload_duration_ns_q48'
```

4.2.4 Phase 3: Main Effects

For each loop, the main effect is the average metric change when the loop transitions from off to on, marginalised over all other loop states:

```
main_effects = {}
for loop in factor_cols:
    on_mean  = df[df[loop] == 1][metric].mean()
    off_mean = df[df[loop] == 0][metric].mean()
    effect   = on_mean - off_mean
    pct      = 100.0 * effect / off_mean
    main_effects[loop] = {'effect': effect, 'pct': pct}

for loop, e in sorted(main_effects.items(),
                      key=lambda x: abs(x[1]['pct']), reverse=True):
    print(f"{loop:40} {e['pct']:+7.2f}%")
```

4.2.5 Phase 4: Two-Way Interactions

The interaction effect for loops a and b:

$$\hat{\beta}_{ab} = \bar{y}_{(1,1)} - \bar{y}_{(1,0)} - \bar{y}_{(0,1)} + \bar{y}_{(0,0)} \tag{4.3}$$

A positive $\hat{\beta}_{ab}$ indicates synergy; negative indicates suppression.

```
from itertools import combinations

for loop_a, loop_b in combinations(factor_cols, 2):
    y_00 = df[(df[loop_a]==0)&(df[loop_b]==0)][metric].mean()
    y_01 = df[(df[loop_a]==0)&(df[loop_b]==1)][metric].mean()
    y_10 = df[(df[loop_a]==1)&(df[loop_b]==0)][metric].mean()
    y_11 = df[(df[loop_a]==1)&(df[loop_b]==1)][metric].mean()
    interaction = y_11 - y_10 - y_01 + y_00
    if abs(interaction) > 100_000:
        print(f"{loop_a} x {loop_b}: {interaction:+,.0f}")
```

4.2.6 Phase 5: Optimal Configuration Search

```
# Top 10 configurations by mean performance
best = (df.groupby('configuration')[metric]
          .mean()
          .nlargest(10)
          .reset_index())
print(best.to_string())
```

4.2.7 Phase 6: Reporting

The analysis report covers four sections:

1. **Main effects table** — individual loop impacts sorted by effect size with p-values.
2. **Interaction matrix** — 6×6 table of two-way interaction estimates; cells highlighted for synergy or suppression.
3. **Optimal configurations** — top 3–5 production candidates on the Pareto frontier of performance versus code complexity.
4. **Statistical summary** — variance by configuration, 95% confidence intervals, regression R^2.

4.3 Complete 2^6 Factorial DoE — Documentation Map

The 2^6 factorial experiment is documented across four artefacts: a quick-start reference, a full design guide, an analysis workflow, and the run script itself.

4.3.1 Document Overview

Artefact	Purpose
Quick-start reference	Three command options (validation, standard, high-precision), time estimates, and basic troubleshooting. Read time: 5 minutes.
Design guide	Full design rationale, six-loop definitions, configuration naming, expected interaction patterns, and troubleshooting. Read time: 20–30 minutes.
Analysis workflow	End-to-end pipeline: data collection, transfer, statistical analysis (main effects, interactions), optimal-configuration search, and reporting. Python code examples included. Read time: 15–20 minutes.
Run script	Executable script generating all 64 configurations, rebuilding for each, randomising 1,920+ runs, and writing metrics to CSV.

4.3.2 Key Concepts

64 configurations. Each configuration is a unique binary string $L_1L_2L_3L_4L_5L_6$ where each digit is 0 (off) or 1 (on). The space spans from `000000` (all loops off; pure FORTH-79 baseline) to `111111` (all loops enabled).

Randomised execution. All $64 \times 30 = 1{,}920$ scheduled runs are interleaved in a single randomised matrix before collection begins. Grouping runs by configuration would introduce thermal ramp and temporal ordering bias; randomisation eliminates both.

Separation of collection and analysis. Data collection and statistical analysis are strictly separated phases. No configuration adjustments are made mid-collection; doing so would introduce confirmation bias. The CSV is analysed only after all runs are complete.

4.3.3 Execution Timeline

Phase	Activity	Duration
Preparation	Choose option, launch script	5–10 min
Collection	64 builds + 1,920 randomised runs	30 min – 12 hr
Transfer	Copy CSV to analysis environment	5 min
Analysis	Main effects, interactions, optimal search	1–2 hr
Reporting	Summary, visualisations, recommendations	1–2 hr

4.3.4 Design Philosophy

The factorial design was chosen over three alternatives:

- **Incremental tuning** (`000000` → `100000` → `110000` → ⋯) misses interaction effects between loops.
- **Dynamic toggle without rebuild** introduces state contamination across configurations.
- **Partial (fractional) factorial** aliases higher-order interaction terms, obscuring synergies and suppressions.

The complete 2^6 factorial is the minimal design that separates all main effects and all two-way interactions without aliasing.

4.4 2^7 Factorial Design of Experiments

The 2^7 DoE campaign exhaustively tested all 128 combinations of StarForth's seven adaptive feedback loops (L1–L7) to identify optimal configurations and interaction effects. It represents the empirical foundation for the L8 Jacquard mode selector.

Scale: 128 configurations × 300 replicates = 38 400 total runs. Duration approximately 6–8 hours on modern AMD64 hardware. Reference commit: `161a3667`.

4.4.1 Experimental Design

Independent Variables

Loop	Factor	Description	Hypothesis
L1	`HEAT_TRACKING`	Execution frequency tracking	May cause cache thrashing
L2	`ROLLING_WINDOW`	Execution history buffer	Enables pattern detection
L3	`LINEAR_DECAY`	Heat dissipation over time	Prevents stale accumulation
L4	`PIPELINING_METRICS`	Word transition prediction	May add overhead
L5	`WINDOW_INFERENCE`	Adaptive window sizing (Levene's)	Optimizes L2 window
L6	`DECAY_INFERENCE`	Exponential regression on heat	Optimizes L3 decay slope
L7	`ADAPTIVE_HEARTRATE`	Dynamic tick frequency	Reduces overhead when stable

Configuration Encoding

Each configuration is a 7-bit binary number with bit position n controlling loop $L(n+1)$. Configuration C97 (binary `1100001`) enables L1, L5, and L6.

Dependent Variables

Primary: `ns_per_word` (execution time per FORTH word) and `cv` (coefficient of variation as stability metric). Secondary: `window_width`, `decay_slope_q48`, `total_heat`, `hot_word_count`, `prefetch_hits`.

4.4.2 Key Findings (300 Replicates, November 2025)

Loop Effectiveness in Top 5% Configurations

Loop	Effect	Top-5% Prevalence	Recommendation
L1 (Heat)	Harmful	14% enabled	Disable by default
L2 (Window)	Workload-dependent	57% enabled	L8-controlled
L3 (Decay)	Beneficial	57% enabled	L8-controlled
L4 (Pipeline)	Harmful	0% enabled	Disable by default
L5 (Window Inf.)	Beneficial	43% enabled	L8-controlled
L6 (Decay Inf.)	Workload-dependent	57% enabled	L8-controlled
L7 (Heartrate)	Beneficial	71% enabled	Always on

Top Configurations

Rank	Config	Binary	Loops	Character
1	C97	`0110001`	L1+L5+L6	Temporal+diverse
2	C7	`0000111`	L3+L5+L6	Full inference
3	C75	`0100011`	L2+L6	Diverse+decay
4	C70	`0100110`	L2+L5+L6	Diverse+inference
5	C1	`0000001`	L1 only	Minimal

Critical insight: No single configuration is optimal across all workload types. This finding directly motivates the L8 Jacquard dynamic mode selector.

4.4.3 Running the Experiment

```
# Generate 128-configuration run matrix
cd experiments/doe_2x7
./generate_run_matrix.sh

# Run full DoE (300 replicates, ~6-8 hours)
./run_doe.sh 300

# Quick test (10 replicates, ~15 minutes)
./run_doe.sh 10
```

Results land in a timestamped directory containing raw CSV data, summary statistics, ANOVA interaction results, and 56 visualization plots (per-loop distributions, pairwise interaction plots, performance heatmap, Pareto frontier).

4.4.4 Analysis

The auto-generated R analysis script (`doe_full_report.R`) produces ANOVA tables with main effects and interactions, Cohen's d and η^2 effect sizes, and Pareto frontiers of speed versus stability. Requires R $\geq$ 4.0 with `ggplot2`, `dplyr`, `tidyr`, and `gridExtra`.

4.5 Workload Shape Validation Experiment

This experiment tests how a single StarForth configuration responds to four distinct workload patterns ("shapes"), validating that the adaptive runtime adjusts appropriately to workload characteristics rather than locking to a fixed operating point.

Configuration tested: C37 (binary `0100101`) — L2 (Rolling Window), L5 (Window Inference), and L7 (Adaptive Heartrate) enabled; all other loops off. **Recommended replicates:** 100–300 per workload. **Reference commit:** `161a3667`.

4.5.1 Workload Shapes

Baseline (`init.4th`). Standard DoE benchmark. Mixed arithmetic and control flow, moderate complexity, predictable pattern. Expected: moderate window size (∼1,024–2,048) and stable heartbeat (∼1 ms ticks).

Square Wave (`init-1.4th`). Alternates between simple and complex operations on every other iteration. Expected: larger window to capture the alternation pattern; more frequent heartbeats during transitions.

Triangle Wave (`init-2.4th`). Gradually increasing then decreasing complexity (ramp up, ramp down). Expected: window size tracking the complexity ramp; heartbeat slowing during stable ramps and accelerating at the inflection point.

Damped Sine (`init-3.4th`). Oscillating complexity that decreases over time via amplitude damping. Expected: window shrinking as amplitude decays; heartbeat rate increasing as the workload stabilizes.

4.5.2 Running the Experiment

```
cd experiments/shape_validation

# Standard validation (100 reps x 4 workloads = 400 runs)
./run_shapes.sh 100

# Quick exploratory (30 reps, ~5 minutes)
./run_shapes.sh 30

# High precision (300 reps x 4 = 1200 runs)
./run_shapes.sh 300
```

Results land in `shape_run_<TIMESTAMP>/shape_results.csv`.

4.5.3 Expected Results

Workload	Expected ns/word	Expected Window	Expected CV
Baseline	∼125	1,024–2,048	∼0.02
Square Wave	∼130+	≥2,048	>0.03
Triangle	∼128	1,536	∼0.025
Damped Sine	∼122	2,048 → 512	∼0.018

Key validation: window width must vary significantly across shapes (ANOVA $p < 0.05$), demonstrating that L5 responds to workload dynamics rather than holding a fixed window.

4.5.4 Success Criteria

1. L5 (Window Inference) produces statistically distinct window widths across shapes (ANOVA $p < 0.05$).
2. Square-wave workload yields a larger window than baseline.
3. Damped-sine workload yields a smaller window than baseline.
4. CV remains below 10% for all shapes (no instability).

Chapter 5

Incompatible Configurations

5.1 Incompatible Configurations in the Factorial Design

5.1.1 Overview

Not all 64 binary combinations of the six feedback loops produce viable builds or stable runtimes. The 2^6 factorial script treats incompatibility as data: failing configurations are recorded in the CSV with status flags and the experiment continues without interruption.

5.1.2 Categories of Incompatibility

Three failure modes are recognised:

1. **Build failure** — the Makefile target produces compilation errors and the binary is not produced.
2. **Runtime crash** — the binary is produced but StarForth exits abnormally (segfault, unhandled signal, or non-zero exit code).
3. **Silent failure** — the binary runs but the test harness produces no metrics output.

Known prerequisite relationships include: L5 (window inference) requires L2 (rolling window) data to operate; L6 (decay inference) requires L3 (linear decay) execution to supply a measurable slope. Combinations that violate these prerequisites may crash or emit no metrics.

5.1.3 Detection and Recording

Build phase. The script invokes `make` for each configuration and inspects the exit code. On failure, the configuration is marked `BUILD_FAILED` and all runs for that configuration are skipped.

Execution phase. For each run, the binary is invoked and its exit code is checked. If the process exits abnormally or no CSV row can be extracted from the output, the run is marked `CRASH`.

CSV columns. Two new columns record compatibility status:

Column	Values	Meaning
`build_status`	`OK`, `BUILD_FAILED`	Makefile compilation result
`run_status`	`OK`, `CRASH`	Runtime exit status

Viable runs carry `OK` in both columns; all metrics fields are populated. Incompatible runs carry the appropriate failure code; metric fields are empty and the loop-flag columns preserve the attempted configuration.

5.1.4 Analysis Filters

```
import pandas as pd
df = pd.read_csv('experiment_results.csv')

# Only viable configurations
viable = df[(df['build_status'] == 'OK') & (df['run_status'] == 'OK')]

# Build failures (per distinct configuration)
build_failures = df[df['build_status'] == 'BUILD_FAILED'].drop_duplicates('configuration')

# Runtime crashes
runtime_crashes = df[df['run_status'] == 'CRASH'].drop_duplicates('configuration')
```

5.1.5 Experiment Continuity

The script does not abort when an incompatibility is detected. A representative run for 64 configurations at 30 replicates yields 1,920 scheduled runs; if 17 configurations are incompatible the output contains approximately 1,847 viable rows alongside 73 failure-flagged rows. The final summary reports both counts.

5.1.6 Interpreting Incompatibility

Incompatible configurations are informative. Patterns — such as L5 appearing consistently in crash reports — indicate unguarded prerequisite relationships in the source code. The recommended response is:

1. Identify the failing loop or combination from the feasibility map.
2. Trace the dependency in source (e.g. L5 reading an uninitialized rolling window when L2 is off).
3. Add a compile-time guard (`#error`) or runtime guard before the affected call site.
4. Re-run the factorial to confirm the crash is eliminated or converted to a clean build failure.

5.1.7 Design Principle

Including incompatibility data rather than suppressing it preserves the full picture of the parameter space. The feasibility map produced by the factorial is itself a contribution: it documents which loop combinations are logically coherent and which impose hidden constraints, informing both production configuration selection and future engineering work.

Chapter 6

Campaign Results — 90 Runs

Figure 6.1: Main effects plot — 90-run campaign (figure pending)

Chapter 7

Campaign Results — 300 Runs

Figure 7.1: Main effects plot — 300-run campaign (figure pending)

Part III

L8 Attractor Map Campaign

Chapter 8

Campaign Design and Execution

8.1 L8 Attractor Map: Session Report, 10 December 2025

This session report documents a six-hour experimental session that began as a memory-safety debugging exercise and ended with the identification of potentially fundamental computational-physics behavior. The empirical findings are current; the theoretical interpretations remain under validation. The 38 580-run empirical basis (38 400 DoE + 180 attractor runs) is the established fact; broader theoretical claims await cross-platform replication.

8.1.1 Memory Safety Fixes

The L8 attractor experiment exhibited a 35–40% non-deterministic crash rate. AddressSanitizer and UndefinedBehaviorSanitizer identified four bugs:

Bug 1 — Heap-buffer-overflow in `dictionary_heat_optimization.c`. A race condition caused `sf_fc_count[bucket_id]` to exceed `sf_fc_cap[bucket_id]`. Fix: exposed the capacity array and added a bounds check with TOCTOU protection before the `memcpy` in `dict_reorganize_buckets_by_heat`.

Bug 2 — Use-after-free in `dictionary_heat_optimization.c`. The heartbeat thread sorted dictionary entry pointers while the main thread freed entries. Fix: acquire `dict_lock` for the entire bucket reorganization pass.

Bug 3 — UBSan left-shift of negative value in M/MOD. Left-shifting a negative signed value is undefined behavior in C99. Fix: cast the dividend to `unsigned long long` before shifting, then cast back.

Bug 4 — UBSan left-shift in Q48.16 conversion. Fix: replace left-shift with multiplication for signed delta values in `src/doe_metrics.c`.

After fixes: 50 consecutive ASan runs, zero errors; 100 production runs, zero crashes; 180-run L8 experiment, 100% success.

8.1.2 Experimental Infrastructure

Workload Identification

Because runs were randomized (to eliminate temporal bias), the heartbeat CSV carried no workload metadata. The solution cross-referenced the run matrix against VM restart markers: heartbeat ticks with `tick_interval_ns` $> 10^9$ indicate VM startup. This yielded a `workload_mapping.csv` cross-reference table without breaking randomization.

Analysis Pipeline

```
cd experiments/l8_attractor_map
./scripts/setup.sh    # Create structure, CSV headers
./scripts/run.sh      # Execute 180 runs, generate mapping
Rscript l8_analysis_safe.R  # Statistics + 13 SVG charts
```

8.1.3 Statistical Results

180 runs (6 workloads × 30 replicates), 4 171 valid heartbeat ticks after filtering.

Workload	Mean (μs)	CV (%)	Character
STABLE	75.45	18.00	Lowest variability
VOLATILE	76.04	22.71	Moderate
OMNI	76.95	26.20	7× compute, comparable stability
TEMPORAL	75.77	32.09	Moderate
DIVERSE	76.87	42.47	High variability
TRANSITION	81.72	69.13	Highest variability (phase-change region)

ANOVA on convergence time: $F(5, 174) = 0.983$, $p = 0.43$ — convergence time is workload-independent, confirming deterministic behavior. ANOVA on tick intervals: $F(5, 4165) = 3.890$, $p = 0.0016$ — workloads produce statistically distinct timing signatures.

8.1.4 Observed Phenomena

All six workloads converged to a ground-state oscillation frequency of approximately $\omega_0 \approx 13.5\,\text{Hz}$ (CV 1.3% across workloads). The damped harmonic oscillator equation $\omega(t) = \omega_0 + Ae^{-\gamma t}\cos(\Omega t + \phi)$ fit the convergence trajectories. Each workload exhibited a distinct effective temperature T_{eff} under a Boltzmann-distribution model of frequency fluctuations (STABLE: $T_{\text{eff}} = 2.175\,\text{Hz}$; TRANSITION: $T_{\text{eff}} = 2.691\,\text{Hz}$). Phase-space portraits revealed 45-degree diagonal relationships across variable pairs, suggesting conserved quantities.

Probability assessments from the session:

- $> 90\%$ confidence: deterministic convergence, workload-independent convergence time, reproducible CV as a metric.
- 70–85% confidence: $\omega_0 \approx 13.5\,\text{Hz}$ is a real pattern; adaptive window finds equilibrium W^*; James Law $K \approx 1.0$; workload spectral signatures are distinct and stable.
- $< 30\%$ confidence: ω_0 is a true universal constant; hardware independence; general computational law.

The frequency likely synchronizes with the CPU clock and is therefore a hardware-dependent emergent property rather than a universal constant.

8.1.5 The Adaptive Window Revelation

A critical finding from this session: `effective_window_size` is *not* logged in the heartbeat CSV. The adaptive window (`src/rolling_window_of_truth.c`) shrinks to 75% when pattern diversity growth falls below 1%, and grows back toward `ROLLING_WINDOW_SIZE` otherwise. The system finds an equilibrium W^* at which it "flatlines." This equilibrium was empirically observed but not yet instrumented; adding `effective_window_size` to the heartbeat CSV was identified as Priority 1 for the next session.

8.1.6 James Law

Empirical observation from this session: $K = \Lambda \times (\text{DoF} + 1)/W \approx 1.0$, where Λ is the effective smoothing factor, W the window size, and DoF the number of active feedback loops. This relationship was named James Law and is under empirical validation in the window-scaling campaign.

8.1.7 Next Validation Steps

1. Add `effective_window_size` logging to heartbeat CSV and re-run 180-run L8 experiment to observe W^* trajectory.

2. Run the window-scaling experiment (2 880 runs; see companion README) to validate James Law across 8 DoF $\times$ 12 window sizes.

3. Cross-platform validation on ARM to test whether ω_0 varies with CPU architecture.

8.2 L8 Attractor Map: Notable Observations from the 9 December 2025 Run

The following observations were recorded during analysis of the 180-run L8 attractor dataset (results_20251209_145907). They represent qualitative findings that supplement the formal statistical report. Several have since been partially explained; others remain open.

The Jitter Mystery. Estimated jitter constituted 86% of the tick interval. This is not measurement noise. The heartbeat operates near a fundamental uncertainty limit: the system cannot be more timing-precise without violating something inherent in its adaptive dynamics.

TRANSITION as a Phase-Change Region. TRANSITION variability (CV $\approx$ 22.5%) was four times higher than STABLE (CV $\approx$ 5%) and produced the most anomalies (10 of 28 total). TRANSITION is not a workload in the ordinary sense; it is a critical point where the system is genuinely unstable, analogous to a melting boundary.

VOLATILE Warms Over Time. Mean tick interval for VOLATILE increased from 74.6 μs (first tick) to 79.9 μs (last tick), a +7.1% rise. This thermodynamic-style accumulation was not expected and gives the name "VOLATILE" an unintended prophetic quality.

Bimodal Hot-Word Distribution. The hot-word count distribution was bimodal: mode at 0 (most ticks have zero hot words) with a secondary cluster at 6–7. The behavior resembles quantum ground-state versus excited-state switching rather than a gradual heating curve.

Golden Ratio Appearance. A tick-interval ratio of approximately 1.583 (close to $\varphi \approx 1.618$) appeared in DIVERSE at tick 13. In chaotic systems the golden ratio appears in bifurcation cascades and resonant frequencies; this single observation is suggestive of self-organized criticality but requires replication before any claim can be made.

Orbiting, Not Settling. Convergence to a stable value took 4–5 ticks, but the system continued oscillating afterward. The L8 attractor finds an orbit around the attractor rather than converging to a fixed point; this is genuine strange-attractor behavior.

Instrumentation Gap. All delta metrics (cache hits, bucket hits, word executions, predictions) were zero throughout the run. The heartbeat was measuring temporal rhythm only, not computational events. This gap was subsequently identified as Priority 1 for the next instrumentation pass.

Fixed Window Width. Window width registered as a constant 4096 throughout the run. This was later understood to be an instrumentation artifact: the adaptive window was changing, but `effective_window_size` was not exported to the heartbeat CSV. Adding this metric was flagged as critical follow-up work.

8.3 L8 Jacquard Mode Selector Validation Experiment

This experiment validates the L8 Jacquard mode selector by comparing dynamic adaptive mode switching against static optimal configurations across five workload types. Its empirical basis is the 2^7 factorial DoE (38,400 runs), which identified L1 and L4 as harmful and established workload-specific optimal configurations for L2, L3, L5, and L6.

8.3.1 Experimental Design

Type: 8×5 factorial design. **Independent variables:** control strategy (8 levels) and workload type (5 levels). **Primary dependent variables:** `ns_per_word` and `cv`.

Control Strategies

Strategy	L1	L2	L3	L4	L5	L6	L7	Description
L8_ADAPTIVE	0	rt	rt	0	rt	rt	1	Dynamic switching
C0_BASELINE	0	0	0	0	0	0	1	Minimal
C4_TEMPORAL	0	0	1	0	0	0	1	Decay only
C7_FULL_INF	0	0	1	0	1	1	1	Full inference
C9_DIVERSE_DECAY	0	1	0	0	0	1	1	Window + decay_inf
C11_DIVERSE_INF	0	1	0	0	1	1	1	Window + inference
C12_DIVERSE_TEMP	0	1	1	0	0	0	1	Window + decay
ALL_ON	0	1	1	0	1	1	1	All except L1/L4

(rt = runtime-selected by L8 mode selector)

Workload Types

Workload	Characteristics	Expected L8 Mode
STABLE	Predictable, repetitive	C0 or C4
DIVERSE	High entropy, mixed operations	C9, C11, or C12
VOLATILE	High CV, random branching	C1 or C7
TEMPORAL	Strong locality, nested loops	C4 or C12
TRANSITION	Phase shifts between workloads	Adaptive

8.3.2 Hypotheses

H1 (Performance). L8_ADAPTIVE matches or exceeds the best static configuration per workload, within a 5% margin.

H2 (Stability). L8_ADAPTIVE shows lower overall CV than any single static configuration across all workloads.

H3 (Adaptation). The L8 mode distribution correlates with workload characteristics.

H4 (Generalization). L8_ADAPTIVE outperforms all static configurations on the TRANSITION workload.

8.3.3 Running the Experiment

```
# Quick test (10 reps = 400 runs, ~8 minutes)
cd experiments/l8_validation
./run_l8_validation.sh 10

# Standard validation (50 reps = 2,000 runs, ~40 minutes)
./run_l8_validation.sh 50

# High precision (100 reps = 4,000 runs, ~80 minutes)
./run_l8_validation.sh 100
```

8.3.4 Analysis

```
Rscript analyze_l8.R l8_validation_YYYYMMDD_HHMMSS
```

Expected plots: ANOVA interaction plot (strategy × workload), L8 mode distribution per workload, Pareto frontier (speed vs. stability), and convergence curves showing mode switches.

8.3.5 Success Criteria

1. L8_ADAPTIVE is at most 5% slower than the best static configuration per workload.
2. L8_ADAPTIVE exhibits the lowest CV across all workloads.
3. L8 mode selections align with expected workload-specific patterns.
4. L8_ADAPTIVE dominates all static configurations on TRANSITION.

Chapter 9

Mathematical Framework

9.1 L8 Attractor: Mathematical Framework for Heartbeat Dynamics

The following equations characterize the temporal behavior of the StarForth heartbeat observed in the 9 December 2025 run. The framework uses quantum-thermodynamic analogs as modeling tools; the equations describe measured phenomena, not claims about quantum physical mechanisms.

9.1.1 Ground State (Equilibrium Frequency)

The equilibrium angular frequency ω_0 was measured independently for each workload:

Workload	ω_0 (Hz)
DIVERSE	13.640 ± 0.916
OMNI	13.430 ± 0.794
STABLE	13.569 ± 0.504
TEMPORAL	13.930 ± 1.237
TRANSITION	13.731 ± 1.978
VOLATILE	13.450 ± 0.949

The CV across all six workloads is 1.3%, indicating a near-universal ground-state frequency of approximately $\omega_0 \approx 13.5\,\text{Hz}$.

9.1.2 Damped Harmonic Oscillator (Convergence Model)

Convergence trajectories fit a damped harmonic oscillator:

$$\omega(t) = \omega_0 + A\, e^{-\gamma t} \cos(\Omega t + \varphi)$$

Fitted parameters for selected workloads:

Workload	γ (tick^{-1})	Ω (rad/tick)	$\tau = 1/\gamma$ (ticks)
DIVERSE	0.725	1.413 (period 4.45 ticks)	1.4
OMNI	0.045	0.450 (period 13.96 ticks)	22.0
STABLE	0.045	0.245 (period 25.67 ticks)	22.4

9.1.3 Boltzmann Distribution (Thermal Fluctuation Model)

Using execution frequency variance as a proxy for thermal energy, frequency fluctuations around ω_0 are modeled by:

$$P(\omega) = \frac{1}{Z} \exp\left(\frac{-(\omega - \omega_0)^2}{k_B T}\right)$$

Effective temperatures T_{eff}:

Workload	$k_B T$ (Hz^2)	T_{eff} (Hz)
STABLE	4.732	2.175
VOLATILE	5.484	2.342
OMNI	5.605	2.367
TEMPORAL	6.678	2.584
TRANSITION	7.240	2.691
DIVERSE	7.483	2.735

9.1.4 Entropy Production Rate

Entropy production was estimated as:

$$\frac{dS}{dt} = \sum_i \frac{\Delta H_i}{T_i}$$

Measured rates ranged from 3.8×10^{-5} to 4.7×10^{-5} (heat units per Hz per tick) across the six workloads.

9.1.5 Frequency–Time Uncertainty

An analog of the Heisenberg uncertainty relation, $\Delta\omega \cdot \Delta t \geq \hbar/2$, was observed in the data. Measured uncertainty products:

Workload	$\Delta\omega \cdot \Delta t$ (Hz·s)
STABLE	3.0×10^{-5}
VOLATILE	4.0×10^{-5}
OMNI	4.8×10^{-5}
TEMPORAL	6.3×10^{-5}
DIVERSE	8.9×10^{-5}
TRANSITION	15.2×10^{-5}

TRANSITION, as the phase-change workload, exhibits the largest uncertainty product — consistent with its high variability and the difficulty of simultaneously characterizing its instantaneous frequency and timing.

9.1.6 Spectral Decomposition

System behavior is modeled as a superposition of normal modes:

$$\omega(t) = \omega_0 + \sum_n A_n \cos(\omega_n t + \varphi_n)$$

Each workload produces a distinct eigenmode spectrum. These spectral signatures provide a basis for workload fingerprinting without explicit runtime classification.

Chapter 10

Heartbeat Analysis

10.1 L8 Attractor Heartbeat Analysis

Date: 9 December 2025. **Experiment:** StarForth L8 Attractor Map. **Runs:** 180 (6 workloads $\times$ 30 replicates). **Total heartbeat ticks:** 4,351.

10.1.1 Key Findings

Workload-Independent Convergence. ANOVA on tick count versus workload: $F(5, 174) = 0.983$, $p = 0.43$. Convergence time is not significantly affected by workload type, confirming deterministic attractor behavior.

Timing Consistency. Mean tick interval: 70–75 μs across all workloads. Coefficient of variation below 5% at steady state.

Workload-Specific Heat Signatures. Each workload exhibits a distinct steady-state word heat signature. VOLATILE showed the highest intra-workload variance ($\sigma = 8.16 \times 10^{-5}$); TEMPORAL, the lowest ($\sigma = 1.76 \times 10^{-5}$).

10.1.2 Conclusions

1. The L8 attractor exhibits deterministic convergence across all six workload types.
2. Heartbeat timing is highly stable (CV $< 5\%$ at steady state).
3. Zero crashes in 180 runs confirm the memory-safety fixes applied prior to this run.
4. The OMNI workload (7$\times$ standard computational load) served as a successful stress test, confirming that timing stability is independent of raw computational intensity.

10.1.3 Output Artifacts

- 13 SVG charts in `analysis_output/charts/`
- 3 CSV tables and ANOVA results in `analysis_output/tables/`

Chapter 11

Comprehensive Results

11.1 L8 Attractor Heartbeat Analysis: Comprehensive Statistical Report

Analysis date: 9 December 2025. **Dataset:** 180 experimental runs (6 workloads × 30 replicates). **Valid heartbeat ticks:** 4,171.

11.1.1 Statistical Summary

The grand mean tick interval was 77.10 μs (median 73.18 μs), with an overall coefficient of variation of 39.91%. Two significance tests both confirmed workload-dependent timing differences:

- ANOVA: $F(5, 4165) = 3.890$, $p = 0.0016$
- Kruskal–Wallis: $H = 17.141$, $p = 0.0042$

Mean ticks to convergence: 23.3 (range 13–39, $\sigma = 2.61$ ticks).

11.1.2 Workload-Specific Performance

Workload	n	Mean (μs)	Std	Min	25%	Median	CV (%)
STABLE	704	75.45	13.58	26.18	71.66	73.09	18.00
VOLATILE	694	76.04	17.27	23.91	71.79	73.17	22.71
OMNI	680	76.95	20.16	29.64	71.94	73.28	26.20
TEMPORAL	706	75.77	24.32	24.30	71.70	73.07	32.09
DIVERSE	716	76.87	32.65	21.82	71.87	73.06	42.47
TRANSITION	671	81.72	56.49	23.95	72.10	73.54	69.13

11.1.3 Interpretation

Deterministic Convergence

All six workloads converged to a ground-state oscillation, confirming that the L8 attractor mechanism is workload-independent in its convergence behavior. The coefficient of variation across convergence times was below 5%, indicating exceptional timing stability once steady state is reached.

Thermal Signatures

Each workload exhibits a characteristic average word heat at steady state, providing a potential fingerprint for workload classification without explicit runtime inspection. TRANSITION's 69% CV distinguishes it from the other five workloads and is consistent with its role as a phase-change boundary rather than a stable operational regime.

Stress Test

OMNI (seven times the computational intensity of a standard workload) produced timing statistics comparable to STABLE, confirming that the adaptive heartbeat mechanism decouples timing from raw computational load.

11.1.4 Files Generated

- 13 SVG charts (distributions, boxplots, time series, heat correlation, violin plots)
- `tick_interval_summary.csv` — descriptive statistics by workload
- `convergence_summary.csv` — tick count analysis by workload

Chapter 12

Binary 2-Cycle Analysis

12.1 Binary 2-Cycle Analysis: Phase-Space Clustering

Date: 11 December 2025. **Data source:** `heartbeat_all.csv` from the L8 Attractor Map experiment.

12.1.1 Method

The analysis applied k-means clustering ($k = 2$) to the (HR, ΔHR) coordinate space, where HR is the tick interval in nanoseconds and ΔHR is the first difference of successive intervals. A binary clarity metric quantifies the separation of the two clusters:

$$\text{BinaryClarity} = \frac{\text{separation}^2}{\text{avg\_spread}}$$

where separation is the Euclidean distance between cluster centers and avg_spread is the average effective radius of the two covariance ellipses. Outliers beyond 5σ were removed before clustering.

12.1.2 Results

Rank	Workload	Binary Clarity	Separation (ns)	Character
1	TRANSITION	1.27×10^6	258,905	Dominant two-regime structure
2	VOLATILE	7.66×10^4	41,487	Moderate separation
3	TEMPORAL	7.02×10^4	44,851	Moderate separation
4	OMNI	6.17×10^4	42,347	Moderate separation
5	STABLE	6.09×10^4	35,932	Weakest separation
6	DIVERSE	6.05×10^4	42,846	Moderate separation

12.1.3 Interpretation

TRANSITION exhibited binary clarity approximately 16× higher than the other five workloads. The two cluster centers were separated by 258,905 ns — a fast-heartbeat regime versus a slow-heartbeat regime — compared to 35,000–45,000 ns for the remaining workloads.

The result is counterintuitive: STABLE shows the *weakest* binary structure despite its name. STABLE operates in a homogeneous regime with tight cluster overlap, while TRANSITION's explicit state switching creates the most pronounced phase-space separation. The 2-cycle structure is present across all workloads; it is most clearly exposed when the workload forces repeated transitions between distinct operational states.

Loop #7 (Adaptive Heartrate) is the likely mechanism: it creates attractor points in phase space by adjusting tick frequency in response to workload characteristics, producing the binary clustering even in nominally non-transitioning workloads.

12.1.4 Open Questions

1. TRANSITION: are exactly two heartbeat regimes present, or is $k = 2$ an oversimplification? Silhouette scores for $k = 3, 4$ are warranted.
2. Autocorrelation analysis of $\mathrm{HR}(t)$ for individual runs would detect periodic switching behavior directly.
3. The Levene's test used internally by Loop #5 (Window Width Inference) should be compared against cluster variance homogeneity to test for cross-loop interaction.

12.1.5 Generated Artifacts

- `binary_2cycle_analysis.svg` — six-panel visualization (1800×1200 px), one panel per workload
- `analyze_2cycle.py` — pure Python analysis script (no external dependencies)

Part IV

Window Scaling Campaign

Chapter 13

Campaign Design

13.1 Window Scaling Experiment: James Law Validation

Objective: Empirically validate James Law:

$$\Lambda = \frac{W}{\text{DoF} + 1}$$

where Λ is the effective smoothing capacity per degree of freedom, W is the rolling window size in bytes, and DoF is the number of active feedback loops (0–7).

13.1.1 Hypothesis

The quantity $K = \Lambda \times (\text{DoF} + 1)/W$ should remain approximately constant across multiple window sizes, multiple degrees of freedom, and diverse workload patterns. $K \approx 1.0$ across all conditions validates James Law as a fundamental scaling relationship in adaptive computational systems.

13.1.2 Experimental Design

Variable	Levels	Values
DoF	8	0–7
Window size	12	512, 1,024, 1,536, 2,048, 3,072, 4,096, 6,144, 8,192, 16,384, 32,769, 52,153, 65,536
Replicate	30	1–30
Total runs		$8 \times 12 \times 30 = 2{,}880$

Fixed workload: `init-l8-omni.4th` (the mega-workload combining all six L8 workload patterns). Run order is shuffled to eliminate temporal bias.

Primary metrics: `ns_per_word`, CV. The K statistic is derived as $K =$ `win_final_bytes`$/(\text{DoF}+1)/W$.

13.1.3 Validation Criteria

- Mean$(K) \in [0.95, 1.05]$
- Std$(K) < 0.1$
- $\max |K - 1.0| < 0.1$

13.1.4 Expected Outcomes

Scenario A — Law holds. $K \approx 1.0$ across all conditions. James Law validated as a universal scaling relationship. Implication: system behavior is predictable and governed by geometric invariants.

Scenario B — Critical threshold exists. Law holds for $W < W_{\text{critical}}$, then degrades. A phase transition is present ("gravitational collapse"). The prior hypothesis places $W_{\text{critical}} \approx$ 16,384 bytes ($4 \times W_0$).

Scenario C — DoF-dependent scaling. K varies systematically with DoF but not randomly. A more complex relationship (logarithmic or power-law) governs the system.

13.1.5 Running the Experiment

```
# Step 1: Generate run matrix
cd scripts/
./generate_run_matrix.R

# Step 2: Pre-build all 96 configurations (~1.5 hours)
./prebuild_all_configs.sh

# Step 3: Execute 2,880 runs (~4-5 hours)
./run_window_sweep.sh

# Step 4: Analyze
./analyze_results.R
```

13.1.6 Analysis Plan

The R analysis script computes:

1. K for all runs and its distribution by condition (DoF $\times$ window)
2. ANOVA to assess significance of DoF and window size effects on K
3. The critical window W_{critical} where CV exceeds a threshold (elbow detection)
4. Five plots: K vs. DoF (faceted by W), K vs. W (faceted by DoF), CV vs. W (phase transition), 3D stability surface, and K-deviation heatmap

13.1.7 Baseline Validation

From prior DoE experiments at the reference window $W_0 = 4{,}096$:

$$\Lambda(\text{DoF}) \times (\text{DoF} + 1) = 4{,}096.0 \pm 0.0 \quad (\text{CV} = 0.00\%)$$

This experiment extends this result to arbitrary window sizes.

13.2 James Law Validation: Quick-Start Guide

Purpose: Validate James Law, $\Lambda = W/(\text{DoF} + 1)$, by running the 2,880-run window-scaling experiment and confirming $K \approx 1.0$ across all conditions.

13.2.1 Prerequisites

```
# Install required R packages
R
> install.packages(c("dplyr", "tidyr", "readr", "ggplot2",
+                    "scales", "viridis", "patchwork"))
> q()
```

13.2.2 Four-Step Workflow (Recommended)

Step 1: Setup and Validation (∼5 minutes)

```
cd experiments/window_scaling_james_law
./setup_experiment.sh
```

Checks R, make, and gcc; generates the shuffled 2,880-row run matrix; runs a test build and a test VM execution; estimates experiment duration.

Step 2: Pre-Build All Configurations (∼1.5 hours)

```
cd scripts/
./prebuild_all_configs.sh
```

Builds all 96 unique (DoF, Window) configurations once and stores them in `experiments/bin/`. Binaries survive `make clean` and can be reused for multiple experiment runs.

Step 3: Execute the Experiment (∼4–5 hours)

```
./run_window_sweep.sh

# Monitor progress in a second terminal:
tail -f ../results/raw/experiment.log
```

Executes 2,880 runs using pre-built binaries. No rebuilds: 3–5 hours faster than the rebuild approach.

Step 4: Analyze Results (∼5 minutes)

```
cd scripts/
./analyze_results.R
```

Computes $K = \Lambda \times (\text{DoF} + 1)/W$ for all runs, tests $K \approx 1.0$, identifies the critical window threshold, and generates five publication-quality plots.

13.2.3 Validation Output

If James Law holds, the analysis script reports:

```
K Statistic Across All Runs:
  Mean(K):      1.0023
  Std(K):       0.0451
  CV(K):        4.50%
  Mean|K-1.0|:  0.0156
  Max|K-1.0|:   0.0823

Validation Criteria:
```

```
  [PASS] Mean(K) in [0.95, 1.05]
  [PASS] Std(K)  < 0.1
  [PASS] Max|K-1.0| < 0.1
```

13.2.4 Timeline

Phase	Duration	Deliverable
Setup	5 min	Environment validated, run matrix generated
Build	1.5 hr	96 configurations built
Execute	4–5 hr	2,880 runs complete
Analyze	5 min	Plots + statistics
Total	∼1 day	James Law confirmed or falsified

Chapter 14

Results and Analysis

Part V

James Law Protocol

Chapter 15

Protocol Definition

15.1 James Law — Window Scaling Experiment Protocol

15.1.1 Objective

The window scaling experiment tests whether the conservation invariant $\Lambda \times (\text{DoF} + 1) = W$ holds across the full range of achievable window sizes, and whether a critical window capacity W^* exists at which the Steady-State Machine undergoes a phase transition.

15.1.2 Conservation Invariant

Empirical validation at $W = 4096$ yielded:

$$\Lambda(\text{DoF}) \times (\text{DoF} + 1) = 4096.0 \quad (\text{CV} = 0.00\%) \tag{15.1}$$

The hypothesis to be tested is that this relationship generalises:

$$\Lambda(\text{DoF}) = \frac{W}{\text{DoF} + 1} \tag{15.2}$$

for all valid window sizes W, and that a critical threshold W^* exists beyond which the invariant breaks down and the SSM collapses to the ground state (all loops off, configuration `0000000`).

15.1.3 Experimental Design

Phase 1 — Coarse sweep. Eight window sizes: 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536 bytes. Five configurations: ground state (`0000000`), best static from 2^7 DoE (`0100011`), L8 adaptive choice (`0110111`), worst static (`1111100`), and `L8_ADAPTIVE`. 100 replicates per (window, configuration) pair; $8 \times 5 \times 100 = 4{,}000$ total runs.

Phase 2 — Critical zone refinement. A fine sweep of seven window sizes within the interval identified as the critical zone by Phase 1, at 200 replicates per pair. Objective: locate W^* within $\pm 1{,}024$ bytes.

Phase 3 — Workload independence validation. Three window sizes centred on W^*, four workload shapes (baseline, damped sine, square wave, triangle), five configurations, 100 replicates. Tests whether W^* is shape-invariant.

Phase	Window sizes	Replicates	Total runs
1 — Coarse sweep	8	100	4,000
2 — Fine refinement	7	200	7,000
3 — Workload validation	3	100	6,000

15.1.4 Key Metrics

- **Stability score** $= 1/\overline{\text{CV}}$ for each window size.
- Λ **deviation** $= |\Lambda_{\text{measured}} - W/(\text{DoF}+1)|$.
- **Collapse probability** $= P(\text{config} = 0 \mid \texttt{L8\_ADAPTIVE})$.
- **Heat density** $= \text{total\_heat}/W$.
- **Variance inflation** $= \text{CV}(W)/\text{CV}(4096)$.

15.1.5 Collapse Threshold Detection

Three independent methods identify W^*:

1. **Mode selection transition.** Plot $P(\text{config} = 0 \mid \texttt{L8\_ADAPTIVE})$ versus W. The threshold is the window size at which this probability crosses 50%.
2. **Variance explosion.** Plot CV versus W. The threshold is where $\text{dCV/dW} \to \infty$ (divergence).
3. **Conservation breakdown.** Plot $\Lambda \times (\text{DoF}+1)$ versus W. The threshold is where the product departs from the conservation value.

Agreement across all three methods constitutes strong evidence for a genuine phase transition.

15.1.6 Success Criteria

1. A reproducible W^* is identified with $\delta W/W^* < 0.2$.
2. $\Lambda \times (\text{DoF}+1) = W$ holds for all $W < W^*$.
3. W^* is independent of workload shape (Phase 3 ANOVA $p > 0.05$).
4. $W^* = kW_0$ for a small integer k, where $W_0 = 4096$.

15.1.7 Planned Extensions

- **Two-dimensional phase diagram.** Vary both W and DoF simultaneously; plot the stability boundary in (W, DoF) space.
- **Hysteresis testing.** Start at $W < W^*$, increase past W^*, decrease back, and test whether the system recovers — analogous to magnetic hysteresis.
- **Adaptive window sizing.** Allow the VM to adjust W dynamically and test whether it self-organises toward $W \approx W_0$.

Part VI

Supplementary Campaigns

Chapter 16

Heartbeat DoE

16.1 Heartbeat DoE — Initial Design

16.1.1 Overview

The heartbeat design-of-experiments experiment constitutes Stage 2 of the StarForth factorial campaign. Where Stage 1 (3,200 runs, six factors, no heartbeat thread) measured static final-performance metrics, Stage 2 incorporates heartbeat observability to capture *temporal stability* characteristics: convergence speed, jitter envelope, load-response coupling, and decay-slope tracking.

16.1.2 Design Points

Stage 2 focuses on the five elite configurations identified by Stage 1 analysis:

#	Configuration	Key Characteristics
1	`1_0_1_1_1_0`	Heat + Decay + Pipelining + Window Inference
2	`1_0_1_1_1_1`	Config 1 plus Decay Inference
3	`1_1_0_1_1_1`	Heat + Rolling Window + Pipelining + both Inferences
4	`1_0_1_0_1_0`	Heat + Decay + Window Inference (lean)
5	`0_1_1_0_1_1`	Rolling Window + Decay + both Inferences (no heat)

16.1.3 New Metrics

Stage 2 extends the `DoeMetrics` struct with approximately 20 heartbeat-specific fields, raising the CSV column count from 38 to approximately 60:

Metric	Type	Purpose	Target
`tick_interval_cv`	double	Jitter coefficient of variation	< 0.15
`tick_outlier_ratio`	double	Fraction of ticks $> 3\sigma$	< 0.02
`decay_slope_convergence_rate`	double	Ticks to slope convergence	> 50
`load_interval_correlation`	double	Workload–heartbeat coupling	> 0.75
`settling_time_ticks`	double	Ticks to 10% settling band	$< 1{,}000$
`overall_stability_score`	double	Composite 0–100 score	> 75

16.1.4 Execution Plan

Five configurations at 50 replicates each yield 250 total runs, randomised into a single test matrix to eliminate ordering bias. The experiment is designed to complete in 2–4 hours.

The composite stability score weights jitter, convergence, and load coupling equally:

$$S = \tfrac{1}{3}(S_{\text{jitter}} + S_{\text{convergence}} + S_{\text{coupling}}) \in [0, 100]$$

The configuration with the highest S becomes the *golden configuration* for use as the MamaForth production baseline.

16.1.5 Note on Subsequent Correction

This initial design assumed a fixed ∼1 ms heartbeat interval and framed the primary metric as jitter in a fixed tick. A subsequent design review identified a critical gap: the heartbeat tick interval `tick_ns` is dynamically modulated by the inference engine. The corrected design measures load-to-heartrate coupling rather than fixed-interval jitter. See Section 16.2.

16.2 Heartbeat DoE — Corrected Design

16.2.1 Critical Discovery

The initial Stage 2 design measured *jitter in a fixed heartbeat interval*, implicitly assuming the tick rate was constant at approximately 1 ms. This assumption is incorrect. The heartbeat tick interval `tick_ns` is a dynamic variable that the inference engine adjusts in response to workload intensity:

```
uint64_t tick_ns;  /* Nanoseconds between ticks -- VARIES with load
   */
```

Stage 1 was conducted with `HEARTBEAT_THREAD_ENABLED=0`, which fixed the interval and disabled the adaptive mechanism entirely. Stage 2 must re-enable the heartbeat thread and measure the quality of its adaptive modulation.

16.2.2 The Adaptive Feedback Loop

The heartbeat acts as a pacing mechanism, not merely a timer. Under high workload, the inference engine increases `tick_ns` — slowing the heartbeat — to allocate more CPU time for physics-parameter tuning. The expected signal chain is:

1. Inference engine detects increased workload intensity.
2. `tick_ns` is increased (heartrate decreases).
3. The physics engine receives more tuning time per tick.
4. The system adapts and eventually converges to a new steady state.
5. `tick_ns` stabilises at the new optimal rate.

16.2.3 Corrected Primary Metric

The primary discriminating metric for Stage 2 is the Pearson correlation between per-tick workload intensity and per-tick heartrate:

$$\rho = \frac{\text{Cov}(W, H)}{\sqrt{\text{Var}(W)\,\text{Var}(H)}} \tag{16.1}$$

where W_t is the dictionary-lookup count in tick t and $H_t = 10^9/\texttt{tick\_ns}_t$ is the heartrate in Hz. A strong positive ρ indicates that the configuration's heartbeat responds proportionally to load. Target: $\rho > 0.75$.

16.2.4 Metric Summary

Metric	Purpose	Target
`load_to_heartrate_correlation`	Load–heartrate coupling	> 0.75
`heartrate_responsiveness_score`	Composite adaptation score	$> 80/100$
`heartrate_convergence_time_ms`	Time to stable rate	$< 5{,}000$ ms
`mean_heartbeat_rate_hz`	Baseline operating rate	$\approx 1{,}000$ Hz
`heartbeat_rate_cv`	Rate variation	0.05–0.20

16.2.5 Implementation Requirements

Four code changes are required:

1. Add `HeartbeatRateSample` struct to `include/vm.h`: fields for tick number, `tick_ns`, workload operations this tick, wall-clock timestamp, and physics state (`hot_word_count`, `total_heat`).
2. Extend the `VM` struct with a pointer to a rate-sample ring buffer and a sample count.
3. Call `capture_heartbeat_rate_sample()` in `heartbeat_thread_main()` before each `nanosleep()`.
4. Extend `DoeMetrics` and `metrics_from_vm()` to compute correlation, responsiveness score, and convergence time from the collected samples.

16.2.6 Expected Discrimination

The corrected design is expected to reveal clear separation among the five elite configurations on the load-correlation axis. A configuration with $\rho < 0.2$ fails to adapt its heartrate to load and should not be selected as the golden configuration regardless of its static performance score.

The golden configuration — the one used as the MamaForth production baseline — is identified as the configuration with the highest composite heartrate responsiveness score, subject to a minimum correlation threshold of $\rho \geq 0.75$.

16.3 Heartbeat DoE — Implementation Summary

16.3.1 Deliverables

The Stage 2 heartbeat DoE implementation produced four artefacts:

1. **Design document** — experiment design with theory, metrics schema, and R analysis template.
2. **DoE run script** — `run_factorial_doe_with_heartbeat.sh`; randomised matrix, incremental builds, per-run logging.
3. **R analysis script** — `analyze_heartbeat_stability.R`; stability ranking, six visualisations, golden-configuration selection.
4. **Execution guide** — pre-execution checklist, step-by-step instructions, metric interpretation, troubleshooting.

16.3.2 Heartbeat Metrics Schema

The `DoeMetrics` struct is extended with 20 heartbeat-specific fields, raising the per-run CSV column count from 38 to approximately 60. Key fields:

Field	Type	Purpose	Target
`total_heartbeat_ticks`	uint64	Ticks during run	—
`tick_interval_mean_ns`	double	Mean interval	≈ 1 ms
`tick_interval_cv`	double	Jitter (CV)	< 0.15
`tick_outlier_ratio`	double	Fraction $> 3\sigma$	< 0.02
`load_interval_correlation`	double	Load–heartrate coupling	> 0.75
`response_latency_ticks`	double	Latency to respond to load	< 50
`settling_time_ticks`	double	Ticks to 10% band	$< 1{,}000$
`overall_stability_score`	double	Composite 0–100	> 75

16.3.3 Composite Stability Score

The overall stability score combines three components equally:

$$S = \tfrac{1}{3}(S_{\text{jitter}} + S_{\text{convergence}} + S_{\text{coupling}}) \tag{16.2}$$

where $S_{\text{jitter}} = 100 - \text{CV} \times 100$, $S_{\text{coupling}} = \rho_{\text{load,interval}} \times 100$, and $S_{\text{convergence}}$ is normalised from the convergence-rate measurement. Target: $S \geq 75$ for production use.

16.3.4 Execution Flow

1. The run script generates a randomised test matrix (250 runs) and prints it for review before prompting for confirmation.

2. For each run: build the configuration (cached if unchanged), execute StarForth with `-doe-experiment`, collect heartbeat and performance metrics, append to CSV.

3. On completion, the R analysis script loads the CSV, computes per-configuration stability rankings, runs pairwise t-tests for significance, generates six visualisations, and prints the golden configuration recommendation.

16.3.5 R Analysis Outputs

```
stability_rankings.csv                          # Per-configuration metric summary
01_stability_scores.png                         # Boxplot ranking
02_jitter_control.png                           # CV comparison
03_convergence_speed.png                        # Convergence-rate comparison
04_load_coupling.png                            # Load-response strength
05_metrics_heatmap.png                          # All metrics normalised
06_tradeoff_jitter_vs_convergence.png           # Trade-off visualisation
```

16.3.6 Golden Configuration Criteria

The golden configuration is identified by the R analysis script as the configuration with the highest composite stability score, subject to $\rho_{\text{load-heartrate}} \geq 0.75$ and a significance threshold of $p < 0.05$ on the pairwise t-test against the second-ranked configuration.

Once identified, the golden configuration is locked into the build system and used as the baseline for all subsequent experiments and as the default physics configuration for MamaForth production builds.

16.4 Heartbeat DoE Phase 2 — Corrected and Finalised

16.4.1 Executive Summary

The initial Phase 2 design measured jitter in a fixed heartbeat interval. The corrected design measures *adaptive heartbeat rate modulation*: how well each configuration's heartbeat rate responds to workload changes. This section is the executive-level statement of the corrected design.

16.4.2 The Physics Story

The heartbeat is not a fixed timer; it is a feedback mechanism. The inference engine adjusts `tick_ns` in response to workload intensity, slowing the heartbeat under high load to allocate more CPU time for physics-parameter tuning:

$$\text{high load} \Rightarrow \uparrow \texttt{tick\_ns} \Rightarrow \text{longer physics tuning window} \Rightarrow \text{convergence to optimal rate}$$

The primary discriminating metric for configuration selection is therefore the Pearson correlation between per-tick workload intensity W_t and per-tick heartrate $H_t = 10^9 \,/\, \texttt{tick\_ns}_t$:

$$\rho = \frac{\mathrm{Cov}(W, H)}{\sqrt{\mathrm{Var}(W)\ \mathrm{Var}(H)}} \tag{16.3}$$

Target: $\rho \geq 0.75$ for a configuration to qualify as a golden-configuration candidate.

16.4.3 How the Initial Design Was Wrong

Aspect	Initial approach	Corrected approach
Primary metric	Jitter in assumed-fixed 1 ms interval	Load-to-heartrate correlation
Thread enabled	No (`HEARTBEAT_THREAD_ENABLED=0`)	Yes (adaptive behaviour active)
Measures	Steady-state smoothness	Adaptive-response quality
Golden criterion	Lowest CV	Highest ρ + fastest convergence

16.4.4 Expected Discrimination

The corrected design is expected to reveal clear ranking separation on ρ. A configuration with $\rho < 0.2$ does not adapt its heartrate to load; regardless of static performance, it is not the golden configuration.

The expected ordering for the five elite configurations (subject to experimental validation):

1. `1_0_1_1_1_0` — balanced loops, expected strongest coupling
2. `1_0_1_1_1_1` — adds decay inference, expected similar coupling
3. `1_1_0_1_1_1` — skips linear decay, alternative adaptation path
4. `1_0_1_0_1_0` — lean configuration, expected slower adaptation
5. `0_1_1_0_1_1` — no heat tracking, expected weakest coupling

16.4.5 Implementation Checklist

The following code changes are required before the experiment can be executed:

- Add `HeartbeatRateSample` struct to `include/vm.h`.
- Add `heartbeat_rate_samples` buffer and sample count to the `VM` struct.
- Track `workload_ops_current_tick` in the execution loop; reset it each heartbeat cycle.
- Call `capture_heartbeat_rate_sample()` in `heartbeat_thread_main()` before each sleep.
- Extend `DoeMetrics` with heartrate fields.
- Implement `analyze_heartbeat_rate_trajectory()` in `src/doe_metrics.c`.
- Update CSV header and row writers.

16.4.6 Analysis Focus

The R analysis script for the corrected design ranks configurations primarily by `load_to_heartrate_correlation`, secondarily by `heartrate_responsiveness_score`:

```
ranking <- doe_data %>%
  group_by(configuration) %>%
  summarize(
    mean_correlation    = mean(load_to_heartrate_correlation),
    mean_responsiveness = mean(heartrate_responsiveness_score)
  ) %>%
  mutate(
    golden_score = (mean_correlation * 0.6) + (mean_responsiveness *
        0.4)
  ) %>%
  arrange(-golden_score)
```

16.4.7 Success Criteria

- Heartbeat rate samples captured per run (sample count > 0).
- CSV contains `load_to_heartrate_correlation` column with non-zero values.
- Correlation metrics span a range of at least 0.4 across the five configurations (sufficient discrimination).
- Golden configuration identified with $p < 0.05$ versus second-ranked.
- Golden configuration achieves $\rho \geq 0.75$.

16.5 Heartbeat DoE Execution Guide

16.5.1 Overview

The heartbeat DoE executes five elite configurations at 50 replicates each (250 total runs) with the heartbeat thread enabled. The experiment requires a clean build environment, approximately 600 MB of storage for run logs and CSV output, and 2–4 hours of uninterrupted machine time.

16.5.2 Pre-Execution Checklist

1. **Build system.** Verify the build completes without errors:

```
make clean
make fastest HEARTBEAT_THREAD_ENABLED=1
./build/amd64/fastest/starforth -c "1␣2␣+␣.␣BYE"
```

2. **Storage.** Confirm at least 1 GB is available in the experiment output directory. The CSV file reaches approximately 50 MB; run logs reach approximately 500 MB.

3. **System stability.** The experiment runs continuously for 2–4 hours. Background CPU load from compilation jobs, browser processes, or automatic system updates biases heartbeat measurements. Minimise competing load before launching.

4. **Heartbeat symbols.** Confirm the heartbeat thread is compiled in:

```
nm ./build/amd64/fastest/starforth | grep heartbeat
```

16.5.3 Execution

```
# Standard run (50 replicates per configuration, ~2-4 hours)
./scripts/run_factorial_doe_with_heartbeat.sh HEARTBEAT_TOP5

# Extended run (100 replicates, ~6-8 hours)
./scripts/run_factorial_doe_with_heartbeat.sh --runs-per-config 100
    HEARTBEAT_EXTENDED
```

The script prints a configuration manifest and a preview of the randomised test matrix before execution begins, then prompts for confirmation. Each run takes approximately 10–15 seconds.

16.5.4 Output Verification

On completion, verify the output file:

```
wc -l experiment_results_heartbeat.csv
# Expected: 251 (header + 250 data rows)

head -1 experiment_results_heartbeat.csv
# Should include: total_heartbeat_ticks, tick_interval_mean_ns,
#   tick_interval_cv, load_interval_correlation,
    overall_stability_score
```

16.5.5 Analysis

```
Rscript scripts/analyze_heartbeat_stability.R \
    /path/to/experiment_results_heartbeat.csv
```

The analysis script produces six visualisations and a `stability_rankings.csv` summary. The golden configuration is the one with the highest composite stability score, subject to $\rho_{\text{load-heartrate}} \geq 0.75$.

16.5.6 Metric Interpretation

Metric	Target	Interpretation
Stability score	$\geq 75/100$	Production-ready configuration
Jitter CV	< 0.15	Heartbeat variation within 15% of mean
Load correlation	> 0.75	Heartrate responds strongly to workload
Convergence	$< 5{,}000$ ticks	Adapts within the test window

A configuration scoring below 65 on any primary axis was likely already excluded by Stage 1 analysis. The golden configuration emerges from clear statistical separation at the top of the stability ranking.

16.5.7 Troubleshooting

All configs score identically. The heartbeat thread may have been compiled out. Verify `HEARTBEAT_THREAD_ENABLED=1` at build time and confirm non-zero values in the `tick_interval_*` columns.

Runtime crashes for specific configurations. Examine the per-run log for the failing configuration. Common causes are thread-safety issues in heartbeat metrics collection or unguarded rolling-window access. See Section 5.1 for the incompatibility handling framework.

R package errors. Install required packages before running the analysis script:

```
1 install.packages(c("tidyverse", "ggplot2", "gridExtra"))
```

Chapter 17

Physics Optimization DoE

17.1 Physics Optimization DoE Guide

17.1.1 Overview

Five optimisation opportunities in the StarForth physics engine are tested sequentially using progressive design-of-experiments: each opportunity explores 3–4 parameter variations at 60 samples per configuration (two iterations of 30 samples), and the winner is locked in before proceeding to the next opportunity.

17.1.2 Q48.16 Fixed-Point Format

All metrics are reported in Q48.16 fixed-point representation: 48 integer bits and 16 fractional bits. This format is deterministic, free of floating-point rounding, and formally verifiable.

Decimal value	Q48.16 integer	Conversion
0.2	13107	$0.2 \times 65536 = 13107.2$
0.33	21627	$0.33 \times 65536 = 21626.9$
0.5	32768	$0.5 \times 65536 = 32768$
0.7	45875	$0.7 \times 65536 = 45875.2$

To convert a Q48.16 CSV value to a decimal: divide by 65536. For example, `vm_workload_duration_ns_c` = 315,797,667,840 corresponds to 315,797,667,840/65536 ≈ 4,822,021 nanoseconds.

17.1.3 Optimisation Opportunities

Opportunity 1 — Decay Slope Inference. Four decay slope values in Q48.16 are tested: 13107 (0.2), 21627 (0.33, baseline), 32768 (0.5), 45875 (0.7). The decay slope controls how rapidly execution heat dissipates from the hot-words cache. Expected improvement: 8–15%. Decision criterion: highest cache hit rate combined with lowest workload duration.

Opportunity 2 — Variance-Based Window Width Tuning. Three rolling window sizes are tested: 2048, 4096 (baseline), 8192 bytes. Window size determines how many execution events are retained for adaptive decisions. Expected improvement: 6–12%. Decision criterion: best balance of context prediction accuracy and workload duration.

Opportunity 3 — Decay Rate Parameter Tuning. Three combinations of decay interval (nanoseconds) and adaptive shrink rate are tested: fast/fast (500 ns, shrink 50), normal/normal (1000 ns, shrink 75, baseline), slow/slow (2000 ns, shrink 100). Expected improvement: 3–6%.

Opportunity 4 — Window × Decay Interaction. A 2×2 factorial crossing two window sizes (2048, 8192) with two decay slopes (0.33, 0.5) reveals interaction effects between the two parameters. Expected improvement: 5–8%.

Opportunity 5 — Hot-Words Cache Threshold. Four cache-promotion thresholds are tested: 5, 10 (baseline), 20, 50 execution-count units. Lower thresholds promote more words to the cache; higher thresholds produce a leaner, higher-precision cache. Expected improvement: 2–4%.

17.1.4 Execution

```
# Opportunity 1: Decay Slope (4 configs x 60 runs = 240 total)
./scripts/run_optimization_doe.sh --opportunity 1 OPP_01_DECAY_SLOPE

# Opportunity 2: Window Width (3 configs x 60 runs = 180 total)
./scripts/run_optimization_doe.sh --opportunity 2
   OPP_02_WINDOW_WIDTH

# ... opportunities 3-5 follow the same pattern
```

All five opportunities complete in approximately 9 minutes at two iterations. To increase statistical power, add `-exp-iterations 4` (doubles samples per configuration).

17.1.5 Analysis

For each opportunity, sort the CSV by `vm_workload_duration_ns_q48` ascending to identify the fastest configuration:

```
tail -n +2 experiment_results.csv | sort -t',' -k27 -n | head -5
```

The winner is then locked into the Makefile before proceeding to the next opportunity. After all five opportunities are completed, a validation run confirms the cumulative improvement over the pre-optimisation baseline.

17.1.6 Expected Cumulative Improvement

Sequential optimisation of all five opportunities is expected to yield a 15–30% cumulative reduction in `vm_workload_duration_ns_q48` compared to the untuned default configuration.

17.2 Physics Engine Validation — Execution Guide

17.2.1 Overview

The comprehensive physics engine validation experiment runs 90 total measurements across three configurations at 30 replicates each. The benchmark executes 100,000 dictionary lookups per run; estimated wall-clock time is 2–3 hours including three separate clean builds.

Configuration	Flags	Purpose
A_BASELINE	`ENABLE_HOTWORDS_CACHE=0 ENABLE_PIPELINING=0`	Control
B_CACHE	`ENABLE_HOTWORDS_CACHE=1 ENABLE_PIPELINING=0`	Cache only
C_FULL	`ENABLE_HOTWORDS_CACHE=1 ENABLE_PIPELINING=1`	Full physics

17.2.2 Execution

```
# Run with default output directory
./scripts/run_comprehensive_physics_experiment.sh

# Run with explicit output path
./scripts/run_comprehensive_physics_experiment.sh ./physics_results

# Analyse results when complete
python3 scripts/analyze_physics_experiment.py \
    ./physics_results/experiment_results.csv \
    --output analysis_report.md
```

Monitor progress while running:

```
watch -n 5 'wc -l physics_results/experiment_results.csv'
# Expected: 91 rows when complete (1 header + 90 data)
```

17.2.3 Output Schema

The CSV contains one row per run. Key columns:

Column	Type	Notes
`configuration`	string	A_BASELINE, B_CACHE, C_FULL
`cache_hit_percent`	float	Hit rate (0–100)
`context_accuracy_percent`	float	Pipelining prediction accuracy
`total_runtime_ms`	float	Wall-clock run time
`speedup_vs_baseline`	float	Computed in analysis script
`ci_lower_95`	float	Lower 95% credible interval
`ci_upper_95`	float	Upper 95% credible interval

17.2.4 Success Criteria

- **A_BASELINE**: All 30 runs complete; consistent execution times; coefficient of variation below 10%.
- **B_CACHE**: Cache hit rate above 20%; speedup 95% credible interval excludes 1.0; CV below 10%.
- **C_FULL**: Prediction accuracy above 60%; speedup exceeds B_CACHE; pattern diversity saturation above 90%.

17.2.5 System Stability

For reproducible measurements: stop background compilation jobs and file indexers before launching; use the `fastest` build profile (default in the script); run in a dedicated terminal with minimal competing I/O.

17.2.6 Publication Notes

Results should be reported as:

- Point estimate (mean speedup).
- 95% credible interval.

- Sample size ($n = 30$ per configuration).
- Hardware description (CPU, RAM, OS, build profile).

Raw CSV and per-run logs should be archived alongside the analysis report for reproducibility.

17.3 Hot-Words Cache Performance Analysis

17.3.1 Summary

The StarForth hot-words cache achieves a statistically confirmed **1.78**× speedup for dictionary lookups, with a 95% Bayesian credible interval of $[1.75\times, 1.81\times]$. All measurements use Q48.16 fixed-point arithmetic; no floating-point computation is required.

17.3.2 Experimental Configuration

- **Build:** `make ENABLE_HOTWORDS_CACHE=1 fastest` (x86_64, full assembly optimisations, LTO, direct threading)
- **Benchmark:** 100,000 dictionary lookups via FORTH test harness; word mix: common FORTH primitives (`IF`, `DUP`, `DROP`, `+`, `-`, `@`, `!`, etc.)
- **Cache:** 32 entries; promotion threshold: 50 executions; eviction: LRU
- **Precision:** Q48.16 64-bit fixed-point (nanoseconds)

17.3.3 Lookup Distribution

Path	Count	Fraction
Cache hits	1,415	35.64%
Bucket hits	1,861	46.88%
Misses	694	17.48%
Total	3,970	100%

17.3.4 Latency Results

Path	Min (ns)	Mean (ns)	Max (ns)
Cache path (1,415 samples)	22	31.543	101
Bucket path (1,861 samples)	23	56.237	370

17.3.5 Speedup Analysis

$$\text{speedup} = \frac{\bar{t}_{\text{bucket}}}{\bar{t}_{\text{cache}}} = \frac{56.237\,\text{ns}}{31.543\,\text{ns}} = 1.78\times \tag{17.1}$$

Time saved per cache hit: $56.237 - 31.543 = 24.694$ ns (43.9% reduction).

Bayesian credible intervals.

Interval	Speedup
95% credible	[1.75×, 1.81×]
99% credible	[1.73×, 1.83×]

$P(\text{speedup} > 1.1\times) = 99.9\%$. All credible-interval arithmetic is performed in pure Q48.16 integer arithmetic; no floating-point or libm functions are used.

17.3.6 Cache Management Behaviour

The physics engine promoted 10 words automatically from the dictionary into the cache based on execution entropy exceeding the threshold of 50. No manual tuning was applied. Zero evictions occurred during the benchmark; the 32-entry cache was sufficient for the active working set.

The two highest-entropy words at experiment completion were `EXIT` (entropy 114) and `LIT` (entropy 101), consistent with their frequency in the compiled FORTH test harness.

17.3.7 Production Properties

- **No floating-point:** All arithmetic is Q48.16 integer; the cache subsystem is compatible with L4Re and bare-metal targets.
- **No external libraries:** No libm dependency.
- **Deterministic:** Near-zero variance across 3,970 samples confirms formally proven deterministic behaviour.
- **Linear complexity:** Cache lookup is O(1); bucket search is linear in bucket depth.

17.3.8 Test Suite Validation

```
make fastest ENABLE_HOTWORDS_CACHE=1
make test
# Total: 782  Passed: 731  Failed: 0  Skipped: 49  Errors: 0
```

All 731 implemented tests pass with the cache-enabled build.

17.4 Physics-Driven Optimisation Proposals

17.4.1 Strategic Direction

The hot-words cache experiment (1.78×, 95% CI: [1.75×, 1.81×]) validates the physics-driven optimisation methodology: collect execution-frequency metrics at runtime, make automatic promotion decisions, and measure impact with statistical rigour. Nine additional opportunities build on the same foundation.

JIT compilation is explicitly excluded from consideration. JIT requires runtime code generation, violates L4Re microkernel policy, introduces floating-point overhead, and defeats formal verification. Physics-driven optimisation achieves measurable gains within StarForth's verification and portability constraints.

17.4.2 Opportunity Catalogue

Opportunity 1 — Return Stack Prediction and Colon Word Inlining. Frequently called colon definitions with small bodies (fewer than 256 bytes) are candidates for compile-time inlining when `execution_heat > 100`. Inlining eliminates dictionary lookup and return-stack overhead. Expected gain: 1.3–2.0× for call-heavy workloads. Verification: static (inlining correctness is provable via Isabelle/HOL).

Opportunity 2 — Block I/O Prefetching. A Markov-style transition matrix tracks which block LBN follows which. When a block is accessed and the next-block heat exceeds a threshold, that block is pre-fetched into the buffer cache. Expected gain: 1.5–3.0× for block-sequential access patterns. Implementation: medium complexity.

Opportunity 3 — Stack Operation Fusion. Co-execution heat tracks consecutive word pairs (e.g., `DUP DROP`, `SWAP ROT`, `OVER SWAP`). Pairs exceeding a heat threshold at compile time are fused into dedicated primitives, eliminating two lookups and one execution per pair. Expected gain: 1.2–1.5× for stack-heavy programs. Verification: static (composition of verified primitives).

Opportunity 4 — Memory Allocation Pattern Prediction. Allocation frequency by size is tracked in a heat vector. Sizes exceeding a threshold trigger pre-warmed pool maintenance. Repeating allocation sequences are detected and cached. Expected gain: 1.3–1.8× for allocation-heavy programs.

Opportunity 5 — Control Flow Branch Prediction. `IF`/`THEN`/`ELSE` outcomes are tracked per source location. Branches with greater than 90% one-sided bias are annotated with a prediction hint that the inner interpreter can use to reorder code or emit x86 branch-hint prefixes. Expected gain: 1.1–1.3× (architecture-dependent).

Opportunity 6 — Vocabulary Search Path Reordering. Per-vocabulary hit rates are tracked. The search order is sorted dynamically by hit rate, placing the most productive vocabulary first. Expected gain: 1.2–1.6× in multi-vocabulary programs. Implementation: low complexity.

Opportunity 7 — String Operation Batching. Consecutive string-output operations (`."`, `EMIT`, `TYPE`) detected at compile time are fused into a single batch output, reducing interpreter loop iterations. Expected gain: 1.1–1.4× for I/O-bound programs. Implementation: low complexity.

Opportunity 8 — Arithmetic Operation Reordering. For hot commutative operations at a given source location, operand sizes are compared and the smaller operand is loaded first to improve prefetch behaviour. Expected gain: 1.05–1.15× (architecture-dependent).

Opportunity 9 — Word Placement Optimisation. Co-execution heat between word pairs guides memory compaction: frequently co-executed words are relocated adjacent to each other in the dictionary to improve instruction-cache locality. Expected gain: 1.05–1.2×. Implementation: high complexity (requires GC integration).

17.4.3 Implementation Roadmap

Phase	Opportunities	Rationale
1 (complete)	Hot-words cache	Proven; production-ready
2 (recommended next)	#3, #6, #1	Highest ROI, lowest complexity
3 (future)	#2, #7, #4	Medium complexity
4 (advanced)	#5, #8, #9	CPU-specific or GC-dependent

17.4.4 Expected Cumulative Gains

Conservative sequential composition of Phases 1–2:

$$1.78\times \times 1.2 \times 1.2 \times 1.3 \approx 3.3\times \text{ cumulative speedup}$$

All nine opportunities together: 5–8× total improvement is plausible, subject to workload dependency and diminishing returns.

17.4.5 Unified Metrics Infrastructure

All nine opportunities require co-execution heat tracking and sequence pattern buffers not yet present in the VM. A one-time extension to the `PhysicsMetrics` per-word structure adds: a co-execution heat vector, timing accumulator, per-word cache-line counters, and a small recent-execution circular buffer. This single investment unlocks the infrastructure required for Opportunities 1–9.

17.5 Reproducing the Hot-Words Cache Experiment

This guide explains how to reproduce the physics-driven hot-words cache optimization experiment, which demonstrated a 1.78× speedup on dictionary lookups in StarForth. The experiment validates the core principle that execution-frequency metrics collected in real time drive automatic optimization decisions, with performance impact measured at statistical rigor using Bayesian inference over Q48.16 fixed-point arithmetic.

Expected results: cache hit rate approximately 35%, speedup factor 1.78× (bucket search versus cache path), 95% credible interval $[1.75\times, 1.81\times]$, mean time saved approximately 348 ns per lookup.

17.5.1 Prerequisites

- x86_64 or ARM64 Linux host with GCC (C99) and `make`
- 512 MB RAM minimum (5 MB VM memory required)
- No external math libraries needed: the physics system uses pure Q48.16 fixed-point arithmetic, making it L4Re-compatible and formally verifiable

17.5.2 Build

Build with the hot-words cache enabled (default):

```
make clean && make
```

Build without the cache to establish a baseline:

```
make clean && make ENABLE_HOTWORDS_CACHE=0
```

Verify the build configuration:

```
./build/amd64/standard/starforth -c "PHYSICS-BUILD-INFO␣BYE"
```

17.5.3 Running the Benchmark

A quick 100 K-iteration run:

```
PHYSICS-RESET-STATS
100000 BENCH-DICT-LOOKUP
PHYSICS-CACHE-STATS
BYE
```

For tighter credible intervals, use 1 M iterations (approximately 60 seconds):

```
PHYSICS-RESET-STATS
1000000 BENCH-DICT-LOOKUP
PHYSICS-CACHE-STATS
BYE
```

17.5.4 Interpreting Key Metrics

Metric	Interpretation
Cache hit rate $> 30\%$	Hot-word identification is functioning
Speedup $> 1.5\times$	Meaningful dictionary acceleration
Cache-path std dev ≈ 0 ns	Deterministic path, correct
Bucket std dev > 0 ns	Normal: varied bucket depths

The Bayesian credible intervals are computed over Q48.16 fixed-point arithmetic. A 95% CI of $[1.75\times, 1.81\times]$ indicates tight, reproducible measurement. $P(\text{speedup} > 1.1\times) \approx 99.9\%$.

17.5.5 Q48.16 Fixed-Point Arithmetic

All timing computations use 64-bit signed Q48.16 fixed-point: 48 integer bits and 16 fractional bits, giving a precision of $2^{-16} \approx 0.0000153$ ns and a range of ± 140 trillion nanoseconds. No floating-point arithmetic is used, preserving L4Re compatibility and formal verifiability.

When the benchmark reports `31.543 ns`: the raw Q48.16 value is $31.543 \times 65536 = 2{,}067{,}029$; divide by 65536 to recover nanoseconds.

17.5.6 Diagnostic Checklist

Symptom	Diagnosis	Remedy
Hit rate $< 20\%$	Dictionary under-exercised	Use `BENCH-DICT-LOOKUP` directly
Speedup $< 1.2\times$	Sample too small	Increase to 100 K+ iterations
High variance	System load	Run in isolation; pin CPU
Cache never fills	Threshold too high	Check `HOTWORDS_EXECUTION_HEAT_THRESHOLD`

17.5.7 References

- Physics implementation: `src/physics_hotwords_cache.c`, `include/physics_hotwords_cache.h`
- Benchmark word definitions: `src/word_source/physics_benchmark_words.c`
- Metrics collection: `src/physics_metadata.c`

Chapter 18

L8 Wiring

18.1 L8 Control-Wiring Plan

Prior to the wiring implementation, the L8 Jacquard Steady State Machine (SSM) computed a 4-bit mode signal and set Boolean flags in `vm->ssm_config`, but the feedback loops L2, L3, L5, and L6 ran unconditionally — the selector was, in the source's words, "a supervisor without authority." This plan defined the four code locations requiring gates and specified the recommended implementation order.

18.1.1 Loop Status Before Wiring

Loop	Name	Pre-wiring status	Control signal
L1	Heat tracking	Compile-time disabled	— (harmful in 86% of configs)
L2	Rolling window	Always on (needed gating)	L8 bit 3 (entropy-driven)
L3	Linear decay	Always on (needed gating)	L8 bit 2 (temporal locality)
L4	Pipelining metrics	Compile-time disabled	— (harmful in 100% of configs)
L5	Window inference	Always on (needed gating)	L8 bit 1 (CV-driven)
L6	Decay inference	Always on (needed gating)	L8 bit 0 (CV + temporal)
L7	Adaptive heartrate	Always on	— (beneficial in 71% of configs)
L8	Jacquard selector	Implemented	4-bit mode controller (C0–C15)

18.1.2 Required Code Changes

Four locations required modification, spanning three source files and approximately 40–60 lines of new conditional logic.

L3 gate — `src/physics_metadata.c:176` Insert an early-return check after the frozen-word and minimum-interval guards in `physics_metadata_apply_linear_decay()`. The gate is internal to the function, keeping call sites unchanged.

L5 gate — `src/inference_engine.c:538` Replace the unconditional call to `find_variance_inflection` with a conditional block that preserves the previous `adaptive_window_width` when L5 is off. A fallback to legacy mode (no `ssm_config`) ensures backward compatibility.

L6 gate — `src/inference_engine.c:546` Symmetric to the L5 gate, guarding `infer_decay_slope_q48(` Previous `adaptive_decay_slope` is preserved when L6 is off.

L2 gate — `src/vm.c:1652` Call-site gate in the hot path of `execute_colon_word()`, wrapping `rolling_window_record_execution()` in a null check on `vm->ssm_config` and a Boolean read. This option (Option A) was preferred over an internal gate inside the rolling-window subsystem for separation-of-concerns reasons.

18.1.3 Implementation Order

The plan recommended proceeding from simplest to most complex:

1. L3 (linear decay) — internal gate, single function.
2. L5 (window inference) — preserve previous value, medium complexity.
3. L6 (decay inference) — symmetric to L5.
4. L2 (rolling window) — call-site gate in the hot execution path.

18.1.4 Validation

Success was defined as all four loops consulting `ssm_config` flags before executing, mode transitions appearing in debug logs, the full test suite passing under the default mode, and the ability to reproduce any of the 16 DoE configurations by forcing L8 to a specific mode.

18.1.5 Open Questions Resolved Post-Implementation

- **Default mode**: The VM initialises at C0 (minimal) and adapts within ∼25 ms; C0 did not break any tests.
- **L2 gate placement**: Option A (call-site) was chosen.
- **Mode override words**: Deferred to a future sprint; listed as planned extensions (`L8-MODE!`, `L8-MODE@`, `L8-AUTO`).

18.2 L8 Control-Wiring Implementation Summary

The L8 Jacquard mode selector was 90% complete before this implementation: the state machine computed a 4-bit mode and set Boolean flags in `vm->ssm_config`, but the execution paths of the four target loops did not consult those flags. This work completed the final "last mile" by adding conditional gates at each loop's execution point.

18.2.1 Changes Made

Four source locations received new gates. The implementation order followed increasing complexity: L3 (single-function internal gate), L5 (medium complexity, preserve previous value), L6 (symmetric to L5), L2 (call-site gate in the hot execution path).

L3 — Linear Decay Gate added to `physics_metadata_apply_linear_decay()` in `src/physics_metadata.c` placed after the frozen-word and minimum-interval checks. When L3 is disabled, words retain heat without decay.

L5 — Window Width Inference Gate added to Phase 2C of `inference_engine_run()` in `src/inference_engine.c` line 538. Previous `adaptive_window_width` is preserved when L5 is off (static configuration).

L6 — Decay Slope Inference Gate added to Phase 2D of `inference_engine_run()` in `src/inference_engine.c` line 563. Previous `adaptive_decay_slope` is preserved when L6 is off.

L2 — Rolling Window Gate added at the call site in `execute_colon_word()` in `src/vm.c` line 1648. No execution history is recorded when L2 is off.

18.2.2 Mode Mapping

The 16 valid L8 modes encode all combinations of the four controlled loops:

Mode	L2	L3	L5	L6	Description
C0	–	–	–	–	Minimal overhead (startup default)
C4	–	✓	–	–	Decay only (top-5% candidate)
C7	–	✓	✓	✓	Full inference without history
C9	✓	–	–	✓	History + slope (top-5% candidate)
C11	✓	–	✓	✓	History + full inference
C12	✓	✓	–	–	DoE-optimal baseline
C15	✓	✓	✓	✓	All loops (maximum adaptivity)

Modes C1–C3, C5–C6, C8, C10, C13–C14 represent intermediate combinations not listed above.

18.2.3 Test Results

```
make clean && make fastest
# Zero warnings, zero errors

make test
# Total: 782  Passed: 731  Failed: 0  Skipped: 49  Errors: 0
```

All 731 implemented tests pass with the default mode C0 (all loops off).

18.2.4 Heartbeat Control Cycle

Each heartbeat tick (∼1 ms) executes in four steps:

1. **Metrics collection** — entropy, CV, temporal decay, and stability score are computed from the rolling window and inference engine outputs.

2. **Mode selection** — `ssm_l8_update()` evaluates the four metrics against thresholds and computes a 4-bit mode with 5-tick hysteresis to suppress rapid flapping.

3. **Configuration application** — `ssm_apply_mode()` decodes the mode bits into the L2/L3/L5/L6 Boolean flags.

4. **Runtime enforcement** — the execution-path gates read the flags at the next loop iteration.

18.2.5 Planned Extensions

Three enhancements are scoped but not yet implemented:

- **FORTH API** — `L8-MODE!`, `L8-MODE@`, and `L8-AUTO` words to support manual mode forcing for DoE validation and benchmarking.
- **Heartbeat CSV column** — add `l8_mode` to the per-tick export so R analysis can correlate mode transitions with performance metrics.
- **DoE re-validation** — run the full 128-configuration experiment with L8 wiring active and confirm 0% algorithmic variance is maintained under dynamic mode switching.

18.3 L8 Control-Wiring Code Review

This section documents the surgical code changes that completed the final "last mile" connection between the L8 Jacquard mode selector and the four adaptive feedback loops it governs (L2, L3, L5, L6). The changes span three source files and approximately 55 lines of new conditional logic.

18.3.1 Summary

All 731 implemented tests passed on a clean `make fastest` build with zero warnings and zero errors. The four execution-path gates were verified individually and in combination.

18.3.2 Files Modified

`src/physics_metadata.c` — L3 Linear Decay Gate

The gate is inserted in `physics_metadata_apply_linear_decay()` after the existing null, frozen-word, and minimum-interval checks, placing it at the optimal early-exit position:

```
/* L8 GATE: Check if L3 (linear decay) is enabled by Jacquard mode
   selector */
if (vm->ssm_config) {
    ssm_config_t *config = (ssm_config_t*)vm->ssm_config;
    if (!config->L3_linear_decay) {
        return;  /* L3 disabled by L8 mode selector */
    }
}
```

When L3 is disabled, words retain execution heat without temporal decay, freezing the thermal state until the mode selector re-enables the loop.

`src/inference_engine.c` — L5 and L6 Inference Gates

Both gates appear in `inference_engine_run()`, one at Phase 2C (window-width inference) and one at Phase 2D (decay-slope inference). Each gate preserves the previous output value when the loop is disabled, providing *sticky configuration* semantics: the last computed value remains in effect rather than resetting to a default.

```
/* L5: Window Width Inference gate */
uint32_t inferred_width = outputs->adaptive_window_width; /* keep
   previous */
if (inputs->vm && inputs->vm->ssm_config) {
    ssm_config_t *config = (ssm_config_t*)inputs->vm->ssm_config;
    if (config->L5_window_inference) {
```

```
        inferred_width = find_variance_inflection(
            trajectory, traj_len, current_variance);
    }
} else {
    inferred_width = find_variance_inflection(  /* legacy mode */
        trajectory, traj_len, current_variance);
}
```

The L6 gate is structurally identical, gating `infer_decay_slope_q48()`. When either inference loop is off, disabling it avoids Levene's test (∼5,000 cycles saved) or exponential regression (∼3,000 cycles saved) on every heartbeat tick.

`src/vm.c` — L2 Rolling Window Gate

The L2 gate sits in the hot path of `execute_colon_word()`, adding a null check and one boolean read (∼2 CPU cycles) per word execution:

```
if (vm->ssm_config) {
    ssm_config_t *config = (ssm_config_t*)vm->ssm_config;
    if (config->L2_rolling_window) {
        rolling_window_record_execution(&vm->rolling_window, word_id
            );
    }
} else {
    rolling_window_record_execution(&vm->rolling_window, word_id);
}
```

Call-site gating (Option A) was chosen over an internal gate inside `rolling_window_record_execution(` to preserve separation of concerns.

18.3.3 L8 Signal Path

The complete control chain from measurement to enforcement is:

1. `vm_heartbeat_update_l8()` computes entropy, CV, temporal decay, and stability score each heartbeat cycle.
2. `ssm_l8_update()` evaluates the four metrics against thresholds and selects a 4-bit mode (C0–C15) subject to 5-tick hysteresis.
3. `ssm_apply_mode()` decodes the mode bits into the L2/L3/L5/L6 boolean flags in `vm->ssm_config`.
4. The execution-path gates (this implementation) read those flags at runtime.

18.3.4 Performance Overhead

Loop	Gate overhead	Saving when disabled
L2 (rolling window)	∼2 cycles	∼10 cycles
L3 (linear decay)	∼2 cycles	∼50 cycles
L5 (window inference)	∼2 cycles	∼5,000 cycles
L6 (decay inference)	∼2 cycles	∼3,000 cycles

Net impact across all four gates is less than 1% overhead; the savings when loops are suppressed are substantial.

18.3.5 Default Behaviour

The VM initialises with mode C0 (all loops off). Within approximately 25 ms the L8 selector adapts to the running workload and enables the appropriate loop subset. The test suite passes in mode C0, confirming that the gates do not break any existing behaviour.

Chapter 19

L8 Validation

Chapter 20

Shape Validation

Appendix A

Reproducibility Protocol

A.1 Reproducibility Protocol

A.1.1 The One-Command Reproduction

The full experimental claim reduces to a single reproducible command:

```
git clone https://github.com/rajames440/StarForth.git && \
cd StarForth && \
make fastest && \
./build/amd64/fastest/starforth --doe --config=C_FULL
```

Expected output: cache hit rate = $17.39 \pm 0.00\%$; runtime approximately 7–10 ms $\pm 60\%$ (hardware-dependent). A deviation in cache CV beyond 0.1% should be filed as a bug.

A.1.2 Exact Reproduction Environment

Docker (Recommended)

The Docker container eliminates all environmental differences:

```
docker build -t starforth-exact -f Dockerfile.exact-reproduction .
docker run --rm \
  -v $(pwd)/reproduction-results:/results \
  starforth-exact
cd reproduction-results/ && sha256sum -c EXPECTED_CHECKSUMS.txt
```

All SHA256 checksums matching constitutes bit-for-bit exact reproduction.

Reference Commit

The experimental baseline is commit `8133787`. Reproduce from this exact state with:

```
git clone https://github.com/rajames440/StarForth.git
cd StarForth
git checkout 8133787
```

Dependency Lock

Canonical environment (from `docker/reproduction.lock`):

Table A.1: Pinned dependency versions for exact reproduction.

Component	Version
OS	Ubuntu 22.04
Kernel	6.2.0-39-generic
GCC	11.4.0-1ubuntu1~22.04
Make	4.3-4.1build1
glibc	2.35-0ubuntu3.8
binutils	2.38-4ubuntu2.6

A.1.3 Reference Hardware

Original experiments conducted on an Intel Xeon Gold 6154 @ 3.00 GHz (18 cores), 128 GB DDR4-2666 ECC, Samsung 970 PRO NVMe 1 TB, Ubuntu 22.04.3 LTS.

Minimum requirements: x86_64 with AVX2 support, 16 GB RAM, 10 GB free disk, Linux kernel 5.x^{+}.

Expected variability:

Table A.2: Tolerances for reproduction on different hardware.

Metric	Tolerance	Reason
Cache CV	0.00% (exact)	Algorithmic determinism
Runtime	±50%	Hardware speed differences acceptable
Convergence rate	±10%	CPU-dependent lookup cost

A.1.4 Environment Configuration

CPU and OS settings critical for matching the baseline:

```
# CPU governor
sudo cpupower frequency-set -g performance

# Disable Turbo Boost (Intel)
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo

# Disable ASLR
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

# Pin to single core
taskset -c 0 ./build/amd64/fastest/starforth --doe
```

These settings eliminate clock-frequency variance, prevent address-layout randomization from perturbing pointer arithmetic, and prevent cache invalidation from core migration.

A.1.5 Full 90-Run Experiment

The complete design-of-experiments protocol (3 configurations × 30 runs = 90 trials) runs via:

```
make reproduce-full-experiment    # ~4 hours
make validate-reproduction        # checks CVs, p-values, checksums
```

Output directory structure:

```
reproduction-results/
  C_NONE/run_{01..30}.csv
```

```
  C_CACHE/run_{01..30}.csv
  C_FULL/run_{01..30}.csv
  summary.txt
  convergence.png
  CHECKSUMS.sha256
```

A.1.6 Statistical Validation

An automated R script verifies the reproduction against expected values:

```
Rscript scripts/validate_reproduction.R \
  --input reproduction-results/ \
  --expected experiment_summary.txt \
  --output validation_report.txt
```

The script checks cache CV (expected 0.00%), convergence p-value (expected < 0.001), and effect size (expected Cohen's $d \approx 5.08$, acceptable $\pm 10\%$), and outputs a PASS/FAIL verdict with specific deviations.

A.1.7 Absence of Hidden Randomness

The implementation contains no random number generators in the production execution path:

```
grep -r "random\|rand\|srand" src/ include/
# Expected: no matches in production code
```

The only acceptable matches are in test code that deliberately introduces randomness to verify failure modes (see §3.1).

A.1.8 Cross-Platform Reproduction

x86_64 (Intel/AMD). Fully supported and validated. Cache CV: 0.00%; convergence: $\approx$ 25%.

AArch64 (ARM, Apple M1). Experimental. Cache CV: 0.00% (determinism holds); convergence magnitude: 18–30% (CPU-dependent).

RISC-V. Untested. Hypothesis: determinism holds (algorithm is architecture-agnostic); performance gains are expected to vary.

A.1.9 Troubleshooting

Cache CV = 12.5%, expected 0.00% Critical failure. Check: `grep -r "rand(" src/` (should return nothing); `cat /proc/sys/kernel/randomize_va_space` (should be 0); `cat /sys/devices/sys` (should be "performance"). If all pass, file a GitHub issue with logs.

Runtime = 150 ms, expected ≈8 ms Debug build used. Run `make clean && make fastest`; confirm `-O3 -march=native -flto` in CFLAGS.

Segmentation fault Run `valgrind -leak-check=full ./build/amd64/fastest/starforth -doe`. File a GitHub issue with full valgrind output.

A.1.10 Long-Term Archival

An archival package including source at commit `8133787`, the full 90-run dataset, the Docker image, the dependency lock file, and this protocol is planned for Zenodo deposit.

A.1.11 Contact

Email `rajames440@gmail.com` (R.A. James) with subject `[StarForth Replication] <brief issue>`. Include `uname -a`, GCC version, build command, error logs, and expected vs. observed results. Response time: within 48 hours.

Appendix B

Replication Invitation

B.1 Invitation to Independent Replication

Independent replication is invited as a scientific obligation, not a courtesy. Complete source code, full experimental data, exact environment specifications, and step-by-step protocols are provided. A failure to reproduce the claimed 0.00% algorithmic coefficient of variation should be reported as a bug; it will be investigated and acknowledged.

B.1.1 Why Replicate

Replication serves different purposes depending on the replicator's starting position. Skeptics who believe the results are implausible can attempt to falsify them; either a bug or an environmental difference will emerge, and both outcomes advance understanding. Researchers who want to build on this work should validate it first: independent replication strengthens evidence, identifies edge cases, and establishes community trust. Researchers working on adaptive systems gain a validated baseline for comparison and a practical demonstration of statistical methods applied to VM tuning.

B.1.2 Replication Levels

Three tiers of replication are defined, from a 15-minute smoke test to a multi-day full reproduction.

Level 1: Smoke Test (15 minutes)

Verifies basic functionality and the core 0% CV claim.

```
git clone https://github.com/rajames440/StarForth.git
cd StarForth
make fastest
./build/amd64/fastest/starforth --doe --config=C_FULL
```

Success criteria: build completes without errors; 780^{+} tests pass; DoE run produces cache CV $\approx 0\%$.

Failure threshold: cache CV $> 0.5\%$.

Level 2: Partial Replication (4 hours)

Reproduces the 25.4% convergence claim across 30 trials.

```
for i in {1..30}; do
  ./build/amd64/fastest/starforth --doe --config=C_FULL \
    > results/run_${i}.csv
```

```
done
python3 scripts/analyze_convergence.py results/
```

Success criteria: all 30 runs show cache CV = 0.00%; late runs show statistically significant improvement ($p < 0.05$); improvement magnitude within $[20\%, 30\%]$.

Failure thresholds: cache CV $> 0.1\%$ in any run; no convergence ($p > 0.05$); improvement $< 10\%$.

Level 3: Full Replication (2–3 days)

Reproduces all five primary claims (C1–C5) across 90 trials.

```
./scripts/full_replication.sh   # 3 configs x 30 runs = 90 runs
Rscript scripts/statistical_validation.R
```

Success criteria: all five claims reproduce within stated error margins; checksums match expected values; statistical tests yield equivalent significance levels.

B.1.3 Exact Reproduction via Docker

A Docker container provides bit-for-bit exact reproduction against a pinned environment, eliminating environmental differences:

```
docker build -t starforth-replication -f Dockerfile.replication .
docker run --rm -v $(pwd)/results:/results starforth-replication
cd results/ && sha256sum -c EXPECTED_CHECKSUMS.txt
```

If all SHA256 checksums match, the reproduction is bit-for-bit identical. If any checksum differs and Docker is used, the discrepancy is in the code rather than the environment.

B.1.4 Manual Environment Configuration

For manual replication outside Docker, the canonical environment is Ubuntu 22.04.3 LTS (kernel 6.2.0-39-generic), GCC 11.4.0, Make 4.3, on an x86_64 system with AVX2 support and 16 GB RAM. Critical configuration steps:

```
# Set CPU governor
sudo cpupower frequency-set -g performance

# Disable Turbo Boost
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo

# Disable ASLR
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space

# Pin to single core
taskset -c 0 ./build/amd64/fastest/starforth --doe
```

Expected variability for correct replication: cache CV is 0.00% (exact match); runtime may differ by $\pm 50\%$ (hardware-dependent); convergence magnitude may differ by $\pm 10\%$ (CPU-specific dictionary lookup cost).

B.1.5 Reporting Results

Successful replications and failures are both scientifically valuable. Report either via GitHub issue, using the tags `replication-success` or `replication-failure`. Include: replicator name and institution, date, replication level, git commit SHA, results summary (cache CV, convergence magnitude, p-value), and hardware and OS specifications.

A response will follow within 48 hours: either an acknowledgment of a bug, diagnostic questions to identify environmental differences, or a request for additional data.

B.1.6 Common Issues and Resolutions

Cache CV = 0.05%, expected 0.00%. The falsification threshold is 0.1%; a CV of 0.05% is within acceptable range. This constitutes a successful replication.

Convergence = 18%, expected 25.4%. The magnitude claim includes ± tolerance; 18% is statistically significant and within range for different hardware. This is a successful replication.

Cache CV = 70%, expected 0.00%. This is a genuine failure. Check in order: clean rebuild (`make clean && make fastest`); ASLR disabled; no `rand()` calls in source (`grep -r "rand(" src/`); valgrind for memory errors. Then file a GitHub issue immediately.

B.1.7 Acknowledgments for Falsification

The first replicator who falsifies Claim C1 (determinism, CV > 0.1% under controlled conditions) will receive co-authorship on any resulting erratum paper. The first replicator who falsifies Claim C2 (convergence, $p > 0.05$ across 30 runs) will receive acknowledgment in future publications. Any replicator who identifies a bug invalidating core claims will receive named credit in the fix commit. Science advances through falsification, and productive falsification is recognized accordingly.

B.1.8 Cross-Institutional Collaboration

Academic laboratories, industry engineering teams, and skeptical researchers make the best replication partners. Technical support (email and video), access to original hardware if needed, and co-authorship on any replication study are available. Contact: `rajames440@gmail.com` (R.A. James).

B.1.9 Replication Checklist

Before beginning:

- Read the executive summary (§**??**)
- Read the formal claim table (§**??**)
- Read the negative results (§3.1)
- Choose a replication level (1, 2, or 3)
- Set up the environment (Docker recommended)

During replication:

- Document all deviations from protocol
- Save all logs and outputs
- Record hardware and software specifications

After replication:

- Compare results to expected values
- Calculate deviations
- File a report (success or failure)

Appendix C

DoE Metrics Schema

C.1 DoE Metrics Schema

Version 1.0, dated 2025-11-19. This schema defines the CSV output produced by every StarForth Design of Experiments run and consumed by R statistical pipelines.

C.1.1 CSV Format

Each run produces a single CSV row:

```
timestamp,configuration,run_number,<35 metrics>
```

Metadata Columns (3)

Column	Type	Description
`timestamp`	ISO 8601 string	Run start time
`configuration`	string	Build configuration label
`run_number`	integer	Sequential run index within configuration

C.1.2 Metrics by Category

Cache Metrics (8 columns)

Column	Type	Range	Description
`total_lookups`	uint32_t	≥ 0	Dictionary lookups in run
`cache_hits`	uint64_t	≥ 0	Hot-words cache hits
`cache_hit_percent`	double	0–100	$100 \times$ hits/lookups
`bucket_hits`	uint64_t	≥ 0	Bucket hits after cache miss
`bucket_hit_percent`	double	0–100	$100 \times$ hits/lookups
`cache_hit_latency_ns`	int64_t	≥ 0	Mean cache-hit latency (ns)
`cache_hit_stddev_ns`	int64_t	≥ 0	Cache-hit latency standard deviation
`bucket_search_latency_ns`	int64_t	≥ 0	Mean bucket-search latency (ns)

All latencies are converted from internal Q48.16 fixed-point to nanoseconds. When `ENABLE_HOTWORDS_CAC` cache columns report zero.

Pipelining Metrics (3 columns)

Populated only when `ENABLE_PIPELINING=1`; measures Loop #4 (word-transition prediction) effectiveness.

Column	Type	Range	Description
context_predictions_total	uint64_t	≥ 0	Prefetch predictions made
context_correct	uint64_t	≥ 0	Correct predictions
context_accuracy_percent	double	0–100	$100 \times$ correct/total

Physics and Adaptive Tuning Metrics (8 columns)

Column	Type	Range	Description
rolling_window_width	uint32_t	256–4096	Effective rolling window size
decay_slope	double	0–1.0	Exponential decay slope
hot_word_count	uint64_t	≥ 0	Words above heat threshold
stale_word_ratio	double	0–1.0	Fraction of stale words
avg_word_heat	double	≥ 0	Mean execution heat, all words
prefetch_accuracy_percent	double	0–100	Speculative prefetch success rate
prefetch_attempts	uint64_t	≥ 0	Total prefetch attempts
prefetch_hits	uint64_t	≥ 0	Successful prefetch hits

Window Tuning Metrics (2 columns)

Column	Type	Range	Description
window_tuning_checks	uint64_t	≥ 0	Adaptive width adjustments
final_effective_window_size	uint32_t	≥ 256	Window size after all tuning

Performance Metrics (3 columns)

All three values are stored in Q48.16 fixed-point format (double = q48_value/65536).

Column	Description
vm_workload_duration_ns_q48	VM workload execution time
cpu_temp_delta_c_q48	CPU temperature change ($\pm 100\,^{\circ}$C)
cpu_freq_delta_mhz_q48	CPU frequency change (± 4000 MHz)

Configuration Knobs (5 columns)

Captured to enable reproducibility analysis: decay_rate_q16, decay_min_interval_ns, rolling_window_size, adaptive_shrink_rate, heat_cache_demotion_threshold.

Feature Flags (2 columns)

enable_hotwords_cache and enable_pipelining (values 0 or 1) identify which optimisations were active during the run. These are the treatment factors in the 2^2 factorial DoE.

C.1.3 R Analysis Template

```
library(tidyverse)

doe_data <- read.csv("experiment_results.csv") %>%
  mutate(
    configuration = as.factor(configuration),
    timestamp = as.POSIXct(timestamp),
```

```
    vm_workload_duration_ms = vm_workload_duration_ns_q48 / 65536 /
        1e6,
    cpu_temp_delta_c = cpu_temp_delta_c_q48 / 65536,
    decay_rate = decay_rate_q16 / 65536
  )

# Factorial model: response ~ cache * pipelining
lm_interaction <- lm(
  vm_workload_duration_ms ~
    as.factor(enable_hotwords_cache) *
    as.factor(enable_pipelining),
  data = doe_data
)
summary(lm_interaction)
```

C.1.4 Statistical Validation Notes

- Minimum runs per configuration: 30; recommended: 150 for correlation analysis.
- Confidence level: 95%.
- Homogeneity of variance tested with Levene's test (`leveneTest()`).
- Detectable effect size at minimum run count: approximately 5% change in any metric.

C.1.5 Version History

Version	Date	Changes
1.0	2025-11-19	Initial schema: 35 metrics, 2^2 factorial DoE support

Appendix D

Methodology Record

D.1 Experimental Methodology — Pre-Publication Record

D.1.1 Design

Three configurations of the StarForth FORTH-79 VM were tested across 90 experimental runs (30 per configuration). The configurations isolated the effect of the physics model by varying exactly one feature at a time:

Configuration	Mechanism	Purpose
A_BASELINE	No adaptation	Control group
B_CACHE	Static word cache	Baseline optimisation comparison
C_FULL	Dynamic physics cache via `execution_heat`	Physics model under test

The benchmark program computed Fibonacci F(20) for 500 iterations. This workload is CPU-bound, deterministic, exercises the dictionary lookup hot path, and accumulates sufficient `execution_heat` to trigger cache promotion.

```
: FIB ( n -- F(n) )
  DUP 2 < IF DROP 1 ELSE
    DUP 1- RECURSE
    SWAP 2 - RECURSE +
  THEN ;

: BENCH 500 0 DO 20 FIB DROP LOOP ;

BENCH BYE
```

D.1.2 Run Protocol

Each run executed: VM initialisation (≈ 0.1 ms), five warmup iterations (not recorded), 30 measurement iterations (all 33 metrics captured), and teardown (≈ 0.1 ms). Fresh VM state between runs; runs executed in sequential blocks (A then B then C) to simplify thermal and time-of-day analysis.

D.1.3 Metrics

Thirty-three metrics were captured per run, including: `RT_AVG`, `RT_MIN`, `RT_MAX`, `RT_STDDEV`, `CACHE_HIT_COUNT`, `CACHE_HIT_RATE`, `DICT_SIZE_FINAL`, `STACK_PEAK_HEIGHT`, `STACK_UNDERFLOW_COUNT`, `THERMAL_CPU_TEMP`, `THERMAL_CPU_FREQ`, `EXECUTION_HEAT_MAX`, `EXECUTION_HEAT_MEAN`, `EXECUTION_HEAT_S`

and 18 additional per-word counters, memory allocation, and block I/O statistics. Total data points: $33 \times 90 = 2{,}970$.

D.1.4 Data Quality

Pre-analysis validation confirmed 91 rows (1 header + 90 runs), zero missing fields, zero out-of-range values, and zero duplicate run IDs. Two minor runtime outliers were retained after IQR analysis confirmed them consistent with OS scheduling variance rather than data corruption. CPU temperature was constant at 27.00°C across all 90 runs, eliminating thermal throttling as a variance source.

D.1.5 Statistical Methodology

Sample size $n = 30$ satisfies the Central Limit Theorem requirement. The 95% confidence interval formula used the conservative approximation $\text{CI}_{0.95} \approx \mu \pm 2\sigma$ (wider than the strict $\mu \pm 1.96\sigma/\sqrt{n}$) to increase reviewer credibility. Shapiro–Wilk tests confirmed near-normal distribution ($p > 0.05$). Durbin–Watson tests showed low autocorrelation. Levene's test confirmed homoscedasticity across configurations.

D.1.6 Reproducibility

```
# Build all three configurations
make                  # A_BASELINE
make fastest         # C_FULL

# Run 90-run experiment
bash scripts/run_comprehensive_physics_experiment.sh \
  physics_experiment/reproduction_test 90

# Analyse
python3 physics_experiment/analyze_variance_sources.py \
  physics_experiment/reproduction_test/experiment_results.csv
```

Hardware: x86-64, 8 GB RAM minimum. OS: Linux kernel 5.10+. Compiler: GCC 9+ or Clang 10+. Python 3.8+ for analysis scripts. Estimated runtime: 30 minutes.

D.2 Variance Decomposition Analysis — Pre-Publication Record

D.2.1 Theoretical Foundation

Total variance decomposes as:

$$\text{Var}(X_{\text{total}}) = \text{Var}(X_{\text{algo}}) + \text{Var}(X_{\text{env}})$$

The strategy is to measure a proxy for algorithmic decisions (cache-hit rate) and a proxy for output (runtime). Zero variance in cache-hit rate with non-zero variance in runtime implies that algorithmic decisions are deterministic while environmental noise drives all observed timing variability.

D.2.2 Per-Configuration Data

Configuration	Metric	Mean	σ	CV
A_BASELINE	RT_AVG (ms)	4.69	3.27	69.58%
	CACHE_HIT_RATE	17.39%	0.00%	**0%**
	EXECUTION_HEAT_MAX	50.47	2.31	4.58%
B_CACHE	RT_AVG (ms)	7.80	2.52	32.35%
	CACHE_HIT_RATE	17.39%	0.00%	**0%**
	EXECUTION_HEAT_MAX	51.23	2.19	4.27%
C_FULL	RT_AVG (ms)	8.91	5.36	60.23%
	CACHE_HIT_RATE	17.39%	0.00%	**0%**
	EXECUTION_HEAT_MAX	63.84	12.11	18.96%

Cache-hit rate is identical (0% CV) across all three configurations and all 90 runs, despite runtime varying between 32% and 70% CV.

D.2.3 Root Cause Analysis

Three hypotheses were ruled out: thermal throttling (CPU held constant at 27.00°C, 0% variance); algorithmic chaos (cache decisions identical every run); and cache instability (hit rate 17.39% ± 0% in every run). The confirmed causes of runtime variance are OS context switching (primary, approximately 50–60%) and memory latency jitter (secondary, approximately 10–20%).

D.2.4 Deployment Implication

Because algorithmic decisions and environmental timing are decoupled, the algorithm can be formally verified in Isabelle/HOL without modelling the OS scheduler or memory system. Environmental load affects when the algorithm completes, not what it decides. This separation enables formal SLAs that guarantee algorithmic behaviour while acknowledging bounded timing uncertainty.

Appendix E

Session Logs

E.1 Experimental Addendum: James Law and Window-Scaling Validation

E.1.1 Discovery Context

During analysis of the SSM attractor results, a conserved geometric relationship was identified in the feedback-response surface of the adaptive runtime. The system exhibits:

- Reproducible convergence to a stable attractor basin.
- Shape-invariant timing behaviour across workloads.
- A consistent geometric structure in the configuration–window–variance phase space.
- A predictable collapse when the smoothing window exceeds a critical bound.

These properties distinguish the StarForth adaptive runtime from chaotic adaptive systems. The behaviour is geometric and predictable, not stochastic.

E.1.2 James Law of Computational Dynamics

The empirically observed scaling relation is stated as follows.

> **James Law.** In an adaptive computational system governed by multiple interacting feedback loops, the effective stability-smoothing factor Λ scales inversely with the number of active degrees of freedom DoF plus one, and directly with the smoothing window size W:
>
> $$\Lambda = \frac{W}{\mathrm{DoF} + 1}$$
>
> This relation describes a conserved geometric structure in the system's feedback-response surface and predicts the onset of stability, metastability, and collapse phases as window size varies.

The law is currently a working hypothesis. Prior to incorporation into any patent application, it is to be validated empirically via the window-scaling experiment described below.

E.1.3 Scientific Significance

If validated, the James Law occupies the same class of discovery as Little's Law in queueing theory, Amdahl's Law in parallelism, and Zipf's Law in linguistics — an empirically observed, falsifiable, reproducible scaling invariant in a new domain (computational dynamics).

E.1.4 Window-Scaling Validation Experiment

The validation experiment tests whether the quantity

$$K = \frac{\Lambda(\text{DoF}) \cdot (\text{DoF} + 1)}{W}$$

remains approximately constant and near unity across multiple degrees of freedom and window sizes.

Parameters

- **Degrees of freedom:** $\text{DoF} \in \{0, 1, 2, 3, 4, 5, 6, 7\}$.
- **Window sizes:** $W \in \{512, 1024, 1536, 2048, 3072, 4096, 6144, 8192, 16384, 32769, 52153, 65536\}$ (12 levels, spanning subcritical through catastrophic-collapse territory).
- **Replicates:** 30 per condition.
- **Total runs:** $8 \times 12 \times 30 = 2{,}880$.
- **Workload:** composite "omni" workload — all L8 initialisation scripts concatenated and run sequentially (STABLE → TEMPORAL → VOLATILE → TRANSITION → DIVERSE), maximally stressing the feedback loops.

Run Matrix

Conditions are generated by a full factorial expansion and shuffled once before execution (fixed random seed for reproducibility) to eliminate temporal drift, thermal bias, and cache-warming artefacts.

Analysis Criteria

1. Compute $K(W, \text{DoF})$ for each condition.
2. Measure mean, standard deviation, and maximum deviation of K from 1.0.
3. Identify the critical window size $W_{\text{crit}}(\text{DoF})$ at which the CV spikes, the L8 engine collapses to a trivial mode, or the attractor structure disappears.

Acceptance Criteria

The law is supported if:

- $K \in [0.97, 1.03]$ across all DoF levels at $W = 4096$.
- The same clustering holds at $W = 2048$ and $W = 8192$.
- Instability (elevated CV or mode collapse) correlates with deviation from $K \approx 1$ at extreme window sizes.

E.1.5 Presentation Guidance

When discussing the James Law in publications, the analogy to physical systems should be used as interpretive framing only, not as a claim. Appropriate language:

> "While the runtime does not implement gravitational physics, the emergent topology resembles a system with a dominant attractor. This analogy is useful as a descriptive shorthand, but the formal behaviour is defined strictly by the equations presented herein."

The experimental evidence — DoE data, convergence plots, variance shrinkage, attractor identification — constitutes the scientific claim. The metaphors motivate the mathematics; they do not replace it.

E.2 Experimental Iteration Runner: Usage Guide

E.2.1 Overview

The `run_doe.sh` script conducts empirical testing across four build configurations in a randomised, aggregated manner.

One experiment equals (30 × iterations) × 4 builds.

Build Configurations

1. **A__BASELINE** — no optimisations (`ENABLE_HOTWORDS_CACHE=0`, `ENABLE_PIPELINING=0`).
2. **A__B__CACHE** — hot-words cache only.
3. **A__C__FULL** — pipelining only.
4. **A__B__C__FULL** — full cache plus pipelining.

All runs are randomised across configurations and aggregated into a single unified dataset.

E.2.2 Interactive Mode

```
# 30x1x4 = 120 runs (~12-15 hours)
./scripts/run_doe.sh ./my_results

# 30x2x4 = 240 runs (~24-30 hours)
./scripts/run_doe.sh --exp-iterations 2 ./my_results

# 30x4x4 = 480 runs (~48-60 hours)
./scripts/run_doe.sh --exp-iterations 4 ./my_results
```

Interactive prompts capture: iteration description and purpose, tuning parameter changes from the previous iteration, expected outcome hypothesis, and prior-iteration observations. The script previews the first 20 lines of the randomised run matrix and waits for confirmation before executing.

E.2.3 CI/CD Mode

Setting the environment variable `ITERATION_NOTES` suppresses all interactive prompts and the confirmation gate:

```
export ITERATION_NOTES="Nightly␣baseline␣validation"
export TUNING_CHANGES="decay_rate_q16=1␣(unchanged)"
export EXPECTED_OUTCOME="Baseline␣determinism:␣variance␣<␣2%"
export PREVIOUS_OBSERVATIONS="Iteration␣1␣showed␣stable␣cache␣
   promotion"

./scripts/run_doe.sh --exp-iterations 2 ./nightly_results
```

E.2.4 Output Structure

```
OUTPUT_DIR/
  experiment_results.csv      -- N rows + header, 33 columns
  experiment_summary.txt      -- metadata, runtime, parameters
  experiment_notes.txt        -- audit trail + iteration context
  test_matrix.txt             -- randomised execution order
  run_logs/                   -- per-run stdout logs
```

E.2.5 Iteration Workflow

A two-iteration study proceeds as follows.

Iteration 1 (baseline): run with `-exp-iterations 1`; analyse with `python3 scripts/analyze_physics_re` observe variance metrics and performance trends; document tuning adjustments.

Iteration 2 (refined): run with `-exp-iterations 2`; provide prior-iteration context via prompts or environment variables; compare CV and performance against iteration 1.

The `experiment_notes.txt` file captures the scientific reasoning for each iteration, creating an audit trail usable for nightly builds and future reference.

E.2.6 Analysis Skeleton

```
import pandas as pd
from scipy import stats

df = pd.read_csv('experiment_results.csv')

# Theorem 1: Determinism
baseline = df[df['configuration'] == 'A_BASELINE']['
    cache_hit_percent']
cv = baseline.std() / baseline.mean() * 100
print(f"Cache hit CV: {cv:.2f}%  (target < 2%)")

# Theorem 2: Performance
full = df[df['configuration'] == 'A_B_C_FULL']['total_runtime_ms']
t, p = stats.ttest_ind(full, baseline)
print(f"Full vs Baseline: t={t:.3f}, p={p:.4f}")

# Theorem 3: ANOVA across all configs
configs = df.groupby('configuration')['total_runtime_ms']
f, p = stats.f_oneway(*[g.values for _, g in configs])
print(f"ANOVA: F={f:.3f}, p={p:.4f}")
```

E.2.7 Troubleshooting

- **Build failure:** inspect `run_logs/build_A_BASELINE.log`.
- **Prompt absent in CI:** set `ITERATION_NOTES` environment variable to suppress interactive mode.
- **Wrong run count:** verify `test_matrix.txt` contains 30 × iterations × 4 lines.
- **Missing CSV rows:** check that the count of files in `run_logs/` matches the expected total runs.

Appendix F

R Script Inventory